EEE3027 Integrated Circuit Design

Semester 1 Report

Sahas Talasila *230057896*

Contents

1 Introduction **3**

2 Transistor Design Labs (Labs 1 & 2) **3**
2.1 NMOS Sweep and Explanation 3
2.2 PMOS Sweep and Explanation 6
2.3 Task 3, Gain Factor Consolidation and Comparison 7
2.4 Lab 2 16

3 FPGA Design Labs (Labs 4, 5 & 6) **26**
3.1 Lab 4, IPSP (Single-Cycle and Pipelined) 26
3.2 Lab 5, DivSub 30
3.3 Task 6, Full Systolic Array Implementation 35

4 Asynchronous Circuit Modelling with Workcraft (Labs 7 & 8) **38**
4.1 Lab 7 - C-Element Modelling and Design 38
4.2 Lab 8 - VME Bus Controller 45

5 References **51**

6 Appendix **51**

Abstract

This is a placeholder for the abstract of the report.

Transistor Design Labs (Labs 1 & 2)

2.1 NMOS Sweep and Explanation

It is important to understand the characteristics and behaviour of NMOS transistors as they are fundamental components in integrated circuit design. In this section, we will discuss the results of the NMOS sweep and provide an explanation of the observed characteristics, as well as the underlying theory behind transistor operation.

A transistor is a semiconductor device used to amplify or switch electronic signals and electrical power. It is composed of semiconductor material, usually with at least three terminals for connection to an external circuit. A voltage or current applied to one pair of the transistor's terminals controls the current through another pair of terminals.

Transistor Structure:

- **Source (S):** The terminal through which carriers enter the channel.
- **Drain (D):** The terminal through which carriers leave the channel.
- **Gate (G):** The terminal that modulates the conductivity of the channel.
- **Body (B):** The substrate on which the transistor is built, often connected to the source.
- **Channel:** The region between the source and drain where current flows when the transistor is on.
- **Oxide Layer:** An insulating layer between the gate and the channel, typically made of silicon dioxide (SiO2).

Figure 1: Basic structure of an NMOS and PMOS transistor.

MOSFET Types

- **N-channel (NMOS):** electrons are charge carriers.
- **P-channel (PMOS):** holes are charge carriers.

Operating Regions of a MOSFET

See the table below for the operating regions of an N-channel MOSFET:

Region	Condition (N-channel)	Description
Cutoff	$V_{GS} < V_{th}$	OFF (no current)
Triode (Linear)	$V_{GS} > V_{th}$, $V_{DS} < V_{GS} - V_{th}$	Acts as variable resistor
Saturation (Active)	$V_{GS} > V_{th}$, $V_{DS} \geq V_{GS} - V_{th}$	Current source (amplifier region)

Code and Explanation

```
* Task 1 Extended: Sweep VDS and VGS (multiple curves)
* nmos_DC_sweep.cir is a netlist.

* Sets the VGS value to 1.2V (can be changed to sweep different values)
.param VGSVAL=1.2

* Tells us the voltages that we will use and the connections (so V_GS
   is a DC source and it has a gate connection (0V DC), same with the
   Drain)
VGS G 0 DC {VGSVAL}
VDS D 0 DC 0
* Tells us the transistor dimensions (gate width (W), channel length (L
   )) along with source and body voltage.
M1 D G 0 0 NLEVEL1 W=10u L=1u

.model NLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02
* VTO is V_th, KP = transconductance parameter, LAMBDA is the channel-
   length modulation factor.

.dc VDS 0 1.2 0.01 VGS 0.4 1.2 0.2
* plots the drain current (I(M1)s) against VDS for different VGS values
    from 0.4V to 1.2V in steps of 0.2V
.plot DC I(M1)s
.end

* Comments start with asterisk
* First line is title (can be anything)
* Last line must be .END (or .end as SPICE is case insensitive)
* Node names: numbers or alphanumeric
* Node 0 is always ground
* Component values: scientific notation (1e-12) or units (1p, 1n, 1u, 1
   m, 1k, 1meg)
```

Listing 1: NMOS Sweep Simulation Code

The netlist defines an NMOS transistor (M1) with width $10\mu m$ and length $1\mu m$. The *.dc* command performs a nested sweep: the primary sweep varies VDS from $0V$ to $1.2V$ in $10mV$ increments, while the secondary sweep steps VGS from $0.4V$ (threshold) to $1.2V$ in $0.2V$ increments.

This generates five $I - V$ characteristic curves, each corresponding to a different gate overdrive voltage. The Level 1 model parameters define the threshold voltage (VTO), transconductance parameter (KP), and channel-length modulation coefficient (LAMBDA).

Results and Discussion

The resulting plot (**Figure 2**) below shows the drain current (I_D) compared to the drain-source voltage (V_{DS}). 5 distinct curves are visible, each one representing a different gate-source voltage (V_{GS}) from $0.4V$ to $1.2V$.

The simulation produces five curves corresponding to $VGS = 0.4V, 0.6V, 0.8V, 1.0V, and 1.2V$.

Each curve exhibits two distinct regions:

Linear Region: For small VDS (where $VDS < VGS - Vth$), the drain current increases approximately linearly with VDS. The transistor behaves as a voltage-controlled resistor.

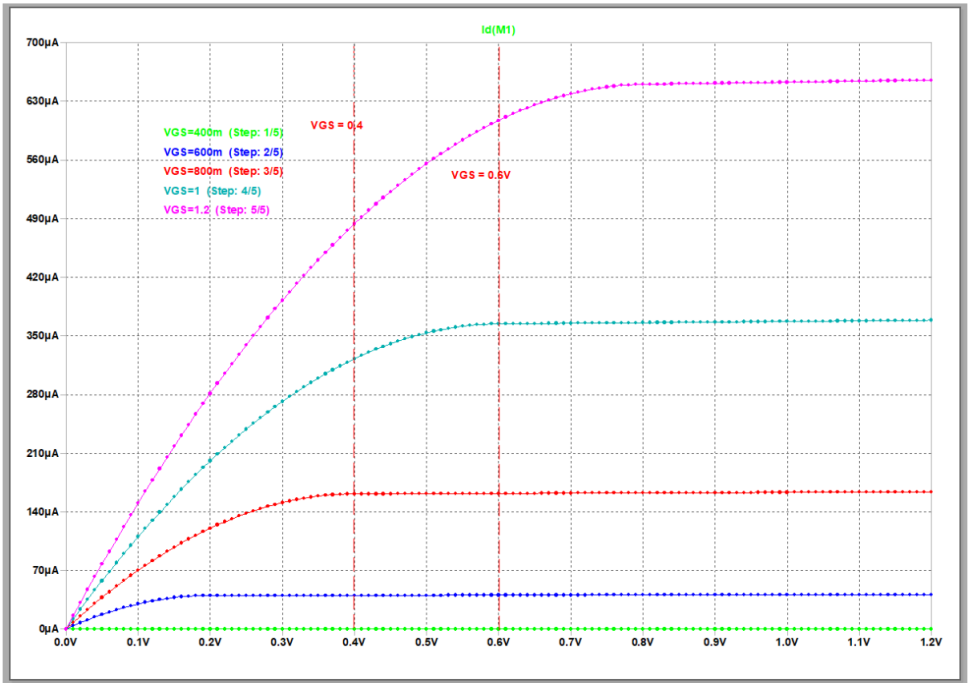

Figure 2: NMOS I-V Characteristics for various V_{GS} values.

Saturation Region: When $VDS \geq VGS - Vth$, the channel pinches off and I_D becomes nearly independent of VDS. The slight upward slope in saturation is due to channel-length modulation ($\lambda = 0.02V^{-1}$). Verification Calculation ($VGS = 1.2V$):

Given: $K_P = 200\,\mu\text{A/V}^2$, $W = 10\,\mu\text{m}$, $L = 1\,\mu\text{m}$, $V_{th} = 0.4\,\text{V}$, and $\lambda = 0.02\,\text{V}^{-1}$.

$$\beta = K_P \frac{W}{L} \tag{1}$$

$$\beta = 200\,\mu \times \frac{10}{1} = 2000\,\mu\text{A/V}^2$$

At saturation with $V_{DS} = 1.2\,\text{V}$, the drain current is

$$I_D = \frac{\beta}{2}(V_{GS} - V_{th})^2(1 + \lambda V_{DS}) \tag{2}$$

Substituting the values,

$$I_D = \frac{2000}{2}(1.2 - 0.4)^2(1 + 0.02 \times 1.2)$$

Evaluating,

$$I_D = 1000 \times 0.64 \times 1.024 = 655.4\,\mu\text{A}$$

This closely matches the simulated value, confirming the accuracy of the model.

The effect of channel-length modulation is observed through the parameter λ. If $\lambda = 0$, the saturation characteristics would be perfectly flat. A non-zero value of $\lambda = 0.02\,\mathrm{V}^{-1}$ introduces a finite output resistance given by

$$r_o = \frac{1}{\lambda I_D}. \tag{3}$$

For a drain current of $I_D = 640\,\mu\mathrm{A}$,

$$r_o = \frac{1}{0.02 \times 640 \times 10^{-6}} \approx 78\,\mathrm{k}\Omega. \tag{4}$$

This effect is significant in analogue circuit design, as a higher output resistance directly improves amplifier gain.

2.2 PMOS Sweep and Explanation

Procedure and Theory For PMOS, all polarities are reversed relative to NMOS. The source connects to the higher potential, and conduction occurs when $VGS < V_{th}$ (where $V_{th} < 0$).

Region	Condition (P-channel)	Description
Cutoff	$V_{GS} > V_{th}$	OFF (no current)
Triode (Linear)	$V_{GS} < V_{th}$, $V_{DS} > V_{GS} - V_{th}$	Acts as variable resistor
Saturation (Active)	$V_{GS} < V_{th}$, $V_{DS} \leq V_{GS} - V_{th}$	Current source (amplifier region)

Code and Explanation

```
* Task 2: PMOS I-V DC Sweep with Multiple VGS Curves
.param VGSVAL=-1.2

VGS G 0 DC {VGSVAL}                  * Negative gate voltage for PMOS
VDS D 0 DC 0

M1 D G 0 0 PLEVEL1 W=10u L=1u        * PMOS transistor

* Level 1 PMOS model (note negative VTO)
.model PLEVEL1 PMOS LEVEL=1 VTO=-0.4 KP=200u LAMBDA=0.02

* Sweep VDS 0 to -1.2V, VGS from -0.4V to -1.2V
.dc VDS 0 -1.2 -0.01 VGS -0.4 -1.2 -0.2

.plot DC I(M1)
.end
```

Listing 2: PMOS Sweep Simulation Code

The netlist mirrors the NMOS structure but with reversed polarities. The *.dc* command sweeps V_{DS} from $0V$ to $-1.2V$ while stepping V_{GS} from $-0.4V$ (threshold) to $-1.2V$ in $-0.2V$ increments. The PMOS model uses VTO $= -0.4V$ to reflect the negative threshold voltage.

Results and Discussion

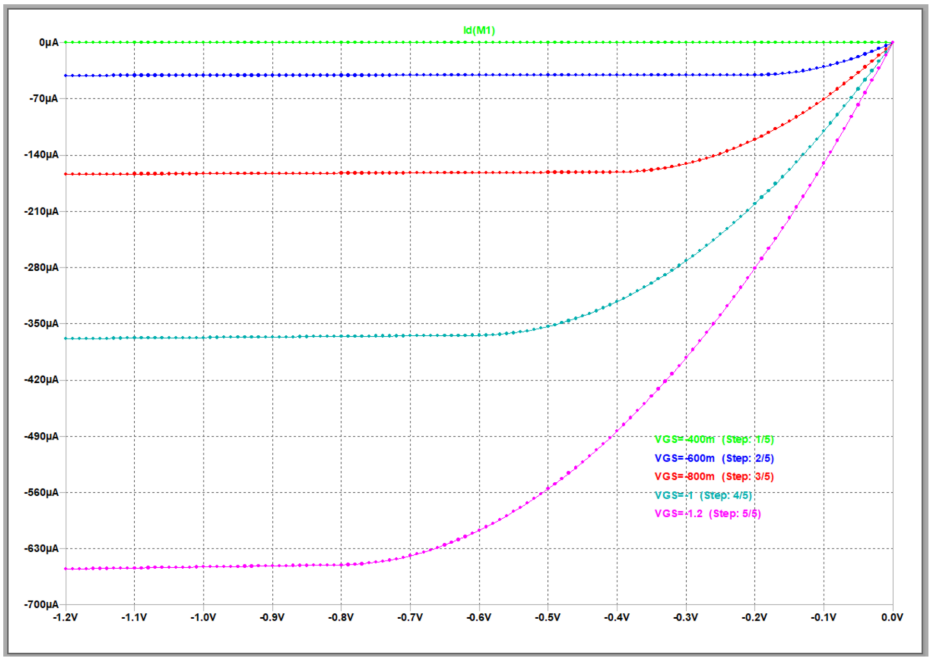

Figure 3: PMOS I-V Characteristics for various V_{GS} values.

From **Figure 3**, we observe five curves corresponding to $VGS = -0.4V, -0.6V, -0.8V, -1.0V$, and $-1.2V$.

The curves mirror NMOS behaviour but appear in the third quadrant. Key observations are as follows.

In the linear region, the drain current increases steeply for small values of $|V_{DS}|$. In the saturation region, the current reaches a plateau when

$$|V_{DS}| \geq |V_{GS}| - |V_{th}|. \tag{5}$$

For verification with $V_{GS} = -1.2\,\mathrm{V}$, the drain current magnitude is

$$I_D = \frac{\beta}{2}(V_{SG} - |V_{th}|)^2 \tag{6}$$

$$I_D = \frac{2000}{2}(1.2 - 0.4)^2 = 640\,\mu\mathrm{A}.$$

The magnitude matches that of the NMOS device because identical K_P and W/L ratios were used. In practical CMOS technologies, PMOS devices typically have a K_P value approximately half that of NMOS due to lower hole mobility. As a result, PMOS transistors are often designed to be approximately twice as wide to achieve matched drive strength.

2.3 Task 3, Gain Factor Consolidation and Comparison

Procedure and Theory

This task investigates how transistor geometry affects the drain current through the gain factor β. From Tasks 1 and 2, the saturation drain current is given by

$$I_D = \frac{\beta}{2}(V_{GS} - V_{th})^2(1 + \lambda V_{DS}). \tag{7}$$

The gain factor β is defined as

$$\beta = \mu_n C_{ox} \frac{W}{L} = K_P \frac{W}{L}. \tag{8}$$

Substituting this expression for β into the saturation current equation yields

$$I_D = \frac{K_P}{2} \cdot \frac{W}{L} \cdot (V_{GS} - V_{th})^2(1 + \lambda V_{DS}). \tag{9}$$

For fixed values of V_{GS}, V_{DS}, and L, this expression simplifies to

$$I_D = k \cdot W, \tag{10}$$

Question 1

From the *.param* lines:

$$VGS = 1.2VVDS = 1.2VL = 1\mu m$$

where k is a constant. This result predicts a linear relationship between the drain current I_D and the transistor width W, which is verified through simulation.

VTO (Threshold Voltage): Symbol: V_T or V_{th}. Units: Volts (V).

Zero-bias threshold voltage the minimum gate-source voltage required to form a conducting inversion channel. Below this voltage, the transistor is in cutoff. Determined by gate material work function, oxide thickness, substrate doping, and interface charges. In our simulation: VTO = 0.4V (NMOS), -0.4V (PMOS).

KP (Transconductance Parameter): Symbol: k' or μC_{ox}. Units: A/V^2.

Intrinsic transconductance parameter representing the product of carrier mobility (μ) and gate oxide capacitance per unit area (C_{ox}):

$$KP = \mu \cdot C_{ox} = \mu \cdot \frac{\varepsilon_{ox}}{t_{ox}} \tag{11}$$

Determines current drive capability. The gain factor is $\beta = (W/L) \cdot KP$, so $I_D \propto KP$. NMOS typically has 2-3× higher KP than PMOS due to higher electron mobility. In our simulation: KP = 200μA/V^2 for both types.

LAMBDA (Channel-Length Modulation): Symbol: λ. Units: V^{-1}. Default: 0.0.

Channel-length modulation coefficient accounting for effective channel shortening as V_{DS} increases in saturation. Without LAMBDA, saturation curves would be perfectly flat (infinite output resistance). With LAMBDA:

$$I_D(sat) = \frac{\beta}{2}(V_{GS} - V_T)^2(1 + \lambda V_{DS}) \tag{12}$$

Output resistance: $r_o = 1/(\lambda I_D)$. Inversely proportional to channel length: $\lambda \propto 1/L$. Analogous to inverse Early voltage in BJTs. In our simulation: $\lambda = 0.02$ V^{-1}, causing $\sim$2.4% current increase across 1.2V V_{DS} range.

Code and Explanation

```
* Task 3: Beta Gain Factor Sweep
.param VGSVAL=1.2 VDSVAL=1.2
.param LVAL=1u

VGS G 0 DC {VGSVAL}
VDS D 0 DC {VDSVAL}

M1 D G 0 0 NLEVEL1 W={WVAL} L={LVAL}

.model NLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02

.step param WVAL 2u 20u 2u          * Sweep W from 2um to 20um
.op                                 * DC operating point analysis

.end
```

Listing 3: PMOS Sweep Simulation Code

The `.step` command varies W while `.op` calculates the DC operating point for each value. Results appear in the SPICE log rather than as a plot.

Question 2B

The standard textbook equation for saturation current is:

$$I_D = \frac{\beta}{2}(V_{GS} - V_T)^2 \quad \text{where } \beta = \frac{W}{L} \cdot \mu_n C_{ox} \tag{13}$$

The complete SPICE Level 1 equation is:

$$I_D = \frac{KP}{2} \cdot \frac{W_{eff}}{L_{eff}} \cdot (V_{GS} - VTO)^2 \cdot (1 + LAMBDA \cdot V_{DS}) \tag{14}$$

To simplify the SPICE model to match textbook equations, the following assumptions are made:

1. LAMBDA = 0 (or very small): The term $(1 + \lambda V_{DS})$ is set to 1, neglecting channel-length modulation. This assumes perfectly flat I-V curves in saturation with infinite output resistance. In our simulation, $\lambda = 0.02$ causes only a $\sim$2.4% variation, which is small but non-zero.

2. No body effect ($\gamma = 0$ or $V_{SB} = 0$): The body effect modifies threshold voltage as:

$$V_T = V_{T0} + \gamma(\sqrt{|2\phi_F + V_{SB}|} - \sqrt{|2\phi_F|}) \tag{15}$$

When bulk and source are both at ground ($V_{SB} = 0$), this term vanishes and $V_T = V_{T0}$, which is our case.

3. Long-channel approximation: Velocity saturation is neglected, assuming drift velocity remains proportional to electric field. Valid for $L \geq 1\mu$m. Short-channel effects (DIBL, threshold roll-off) are ignored.

4. No mobility degradation: Carrier mobility μ is assumed constant, independent of V_{GS}. In reality, high vertical electric fields reduce mobility, but Level 1 ignores this.

5. Ideal effective dimensions: $W_{eff} = W$ and $L_{eff} = L$ (no parasitic effects). When $LD = WD = XL = XW = DEL = 0$, drawn and effective dimensions are equal.

6. Subthreshold conduction ignored: For $V_{GS} < V_T$, Level 1 assumes $I_D = 0$. In reality, exponential subthreshold current exists: $I_D \propto \exp(V_{GS}/nV_t)$.

Verification of equivalence: If $KP = \mu_n C_{ox}$, then:

$$\beta_{textbook} = \frac{W}{L} \cdot \mu_n C_{ox} = \frac{W}{L} \cdot KP = \beta_{SPICE} \tag{16}$$

With $LAMBDA = 0$, $V_{SB} = 0$, and ideal geometry, the equations become identical.

$$I_D = \frac{K_P}{2} \frac{W}{L} (V_{GS} - V_{th})^2, \tag{17}$$

with no body-effect corrections included.

Results and Discussion

```
* --- Fixed Bias Parameters ---
.param VGSVAL=1.2
.param VDSVAL=1.2
.param LVAL= 1u

* --- Voltage Sources ---
VGS G 0 DC {VGSVAL}
VDS D 0 DC {VDSVAL}

* --- NMOS with L=2um (doubled) ---
M1 D G 0 0 NLEVEL1 W={WVAL} L={LVAL}

* --- Model ---
.model NLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02

* --- Sweep W ---
.step param WVAL 2u 20u 2u

* --- Analysis ---
.op
.print OP I(M1)

.end
```

Listing 4: Beta Sweep Simulation Code

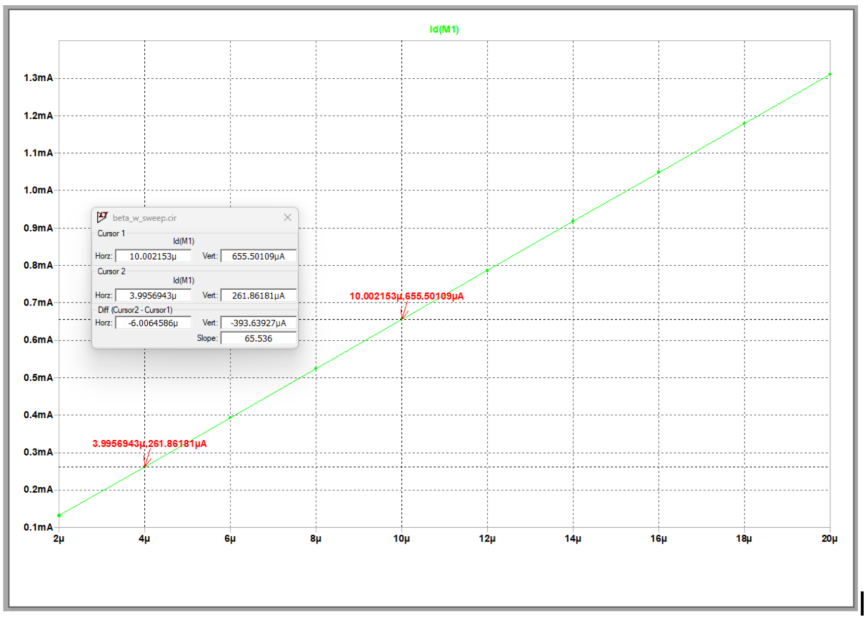

Figure 4: Beta Sweep

Figure 4, shows the linear relationship between drain current I_D and transistor width W. Each data point corresponds to a different width from $2\mu m$ to $20\mu m$.

The theoretical slope can be derived from the saturation drain current expression using the given parameters,

$$I_D = \frac{K_P}{2L}(V_{GS} - V_{th})^2(1 + \lambda V_{DS}) \cdot W. \tag{18}$$

Differentiating with respect to the transistor width W gives the slope,

$$\text{Slope} = \frac{dI_D}{dW} = \frac{K_P}{2L}(V_{GS} - V_{th})^2(1 + \lambda V_{DS}). \tag{19}$$

Substituting the values $K_P = 200\,\mu\text{A/V}^2$, $L = 1\,\mu\text{m}$, $V_{GS} = 1.2\,\text{V}$, $V_{th} = 0.4\,\text{V}$, $\lambda = 0.02\,\text{V}^{-1}$, and $V_{DS} = 1.2\,\text{V}$,

$$\text{Slope} = \frac{200\,\mu}{2 \times 1\,\mu}(1.2 - 0.4)^2(1 + 0.02 \times 1.2). \tag{20}$$

Evaluating,

$$\text{Slope} = 100 \times 0.64 \times 1.024 = 65.54\,\mu\text{A}/\mu\text{m}. \tag{21}$$

From simulation, taking two points such as $W = 20\,\mu\text{m}$ with $I_D \approx 1310\,\mu\text{A}$ and $W = 2\,\mu\text{m}$ with $I_D \approx 131\,\mu\text{A}$,

$$\text{Slope} = \frac{1310 - 131}{20 - 2} = \frac{1179}{18} \approx 65.5\,\mu\text{A}/\mu\text{m}. \tag{22}$$

This closely matches the theoretical prediction, confirming that $I_D \propto W$.

Doubling Channel Length

If the channel length is doubled to $L = 2\,\mu$m, then from the definition of the gain factor,

$$\beta = K_P \frac{W}{L}. \tag{23}$$

The new gain factor becomes

$$\beta_{\text{new}} = K_P \frac{W}{2L} = \frac{\beta_{\text{original}}}{2}. \tag{24}$$

As a result, the drain current I_D is halved for all values of W. Consequently, the slope of the I_D versus W characteristic becomes

$$\text{New Slope} = \frac{65.54}{2} = 32.77\,\mu\text{A}/\mu\text{m}. \tag{25}$$

This result illustrates why shorter channel lengths are preferred in transistor design to achieve higher drive current.

Results and Discussion

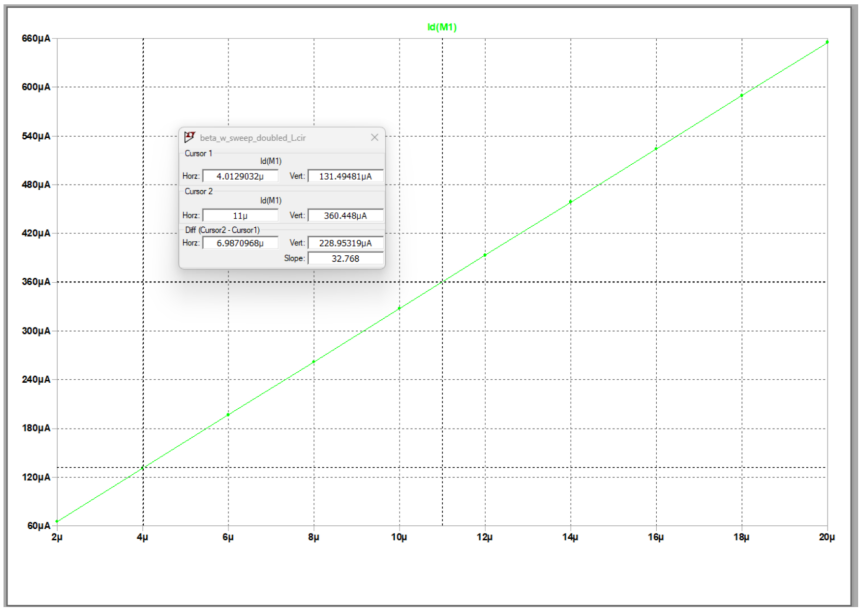

Figure 5: PMOS I-V Characteristics for various V_{GS} values.

Figure 5 shows the linear relationship between drain current I_D and transistor width W when the channel length is doubled to $L = 2\,\mu$m. Each data point corresponds to a different width from $2\mu m$ to $20\mu m$.

Reducing VGS

When $V_{GS} < V_{th}$, for example $V_{GS} = 0.3\,\mathrm{V} < 0.4\,\mathrm{V}$, the transistor operates in the cutoff region. In this condition, no inversion layer is formed and the drain current satisfies

$$I_D \approx 0, \tag{26}$$

independent of the transistor width W. This behaviour explains why the threshold voltage defines the effective on/off boundary in digital switching applications.

Why PMOS is wider than NMOS

In Task 2, equal values of K_P were used for NMOS and PMOS devices. In practice, however, the process transconductance parameters differ:

$$K_{P,n} = \mu_n C_{ox} \approx 200\,\mu\mathrm{A/V}^2, \tag{27}$$

$$K_{P,p} = \mu_p C_{ox} \approx 80\text{–}100\,\mu\mathrm{A/V}^2. \tag{28}$$

Since the hole mobility is lower than the electron mobility, with $\mu_p \approx 0.4\mu_n$, a PMOS transistor must be approximately 2–2.5 times wider than an NMOS transistor to achieve symmetric rise and fall times in a CMOS inverter.

Task 4, Iterative Solutions

SPICE solves circuits containing nonlinear devices, such as diodes and transistors, using iterative numerical methods. Unlike linear circuits, where Kirchhoff's laws lead to direct algebraic solutions, nonlinear device equations require iterative root-finding techniques.

Applying Kirchhoff's Voltage Law (KVL) to the series circuit gives

$$V_s - IR - V_d = 0. \tag{29}$$

The diode current is described by the Shockley equation,

$$I = I_s \left(e^{V_d/nV_t} - 1\right). \tag{30}$$

Substituting this expression for I into the KVL equation yields

$$f(V_d) = V_s - RI_s \left(e^{V_d/nV_t} - 1\right) - V_d = 0. \tag{31}$$

This is a nonlinear equation in V_d and cannot be solved analytically.

To find the root of $f(V_d) = 0$, SPICE employs the Newton–Raphson method, which iterates according to

$$V_d^{(k+1)} = V_d^{(k)} - \frac{f\left(V_d^{(k)}\right)}{f'\left(V_d^{(k)}\right)}. \tag{32}$$

The derivative of $f(V_d)$ is

$$f'(V_d) = -\frac{RI_s}{nV_t} e^{V_d/nV_t} - 1. \tag{33}$$

Convergence is achieved when the change in voltage satisfies $|\Delta V_d| < \varepsilon$, where ε is a chosen tolerance.

In earlier tasks, the Level 1 MOSFET model was used, which also involves nonlinear equations such as

$$I_D = \frac{\beta}{2}(V_{GS} - V_{th})^2. \tag{34}$$

When SPICE computes the DC operating point, it applies the NewtonRaphson method internally to solve these nonlinear device equations simultaneously with the Kirchhoff current and voltage constraints.

Parameter	**Value**
V_s	$5\,\mathrm{V}$
R	$1\,\mathrm{k\Omega}$
I_s	$10^{-12}\,\mathrm{A}$
n	1
V_t	$26\,\mathrm{mV}$ (at $T = 300\,\mathrm{K}$)

Table 1: Circuit and diode parameters used in the analysis

Due to the length of the code, use the link to view the Python script here: (INSERT PYTHON CODE LINK GITHUB HERE).

Figure 6 shows the terminal output from running the NewtonRaphson iterative solver for the diode circuit. The iterations converge to a diode voltage of approximately $0.58\,\mathrm{V}$, yielding a current of about $4.4\,\mathrm{mA}$.

```
(base) sahast@sahast-Zenbook-S-13-UX5304VA-U
27_Project/ICDesign/Code/PythonSolutionCode/

Newton–Raphson Iteration Results:

 Iteration       Vd (V)   Change (V)
         1 6.743050e-01 2.569504e-02
         2 6.489609e-01 2.534402e-02
         3 6.245485e-01 2.441248e-02
         4 6.024855e-01 2.206293e-02
         5 5.856140e-01 1.687148e-02
         6 5.770664e-01 8.547596e-03
         7 5.753132e-01 1.753236e-03
         8 5.752515e-01 6.172715e-05
         9 5.752514e-01 7.324502e-08

Final Results:
Diode voltage (Vd): 0.575251 V
Diode current (I): 4.424749e-03 A
```

Figure 6: Newton Raphson Terminal Output

A cobweb diagram can be used to visualise the convergence of the NewtonRaphson iterations. **Figure 7** illustrates how successive approximations approach the root of the nonlinear equation.

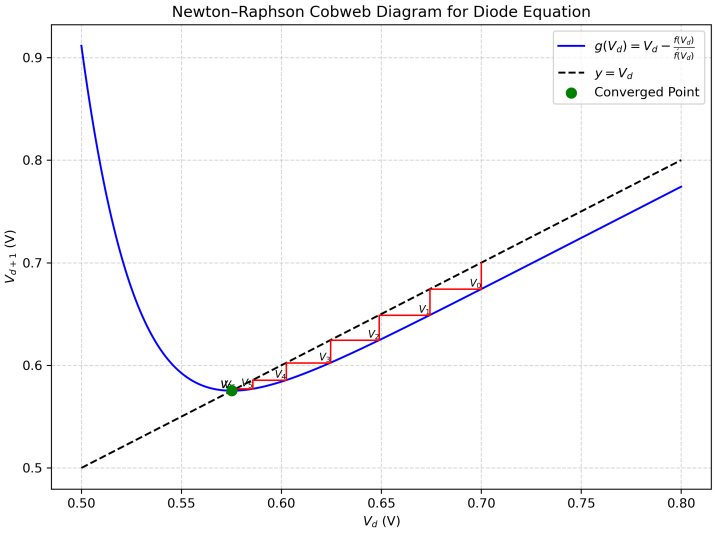

Figure 7: Cobweb Diagram Showing the Convergence of NewtonRaphson Iterations.

Results and Discussion

The Newton–Raphson method converges rapidly due to its quadratic convergence property, meaning that the number of correct digits approximately doubles with each iteration. From the numerical output, six-digit accuracy is achieved in approximately seven iterations. This computational efficiency is the reason SPICE employs the Newton–Raphson algorithm for DC operating-point analysis.

Verification of the solution can be performed using Kirchhoff's Voltage Law,

$$V_s - IR - V_d = 5 - (4.383 \times 10^{-3})(1000) - 0.6166 = 0, \tag{35}$$

which confirms that the computed values satisfy the circuit constraints.

When SPICE analyses the MOSFET circuits from Tasks 1–3, it formulates nonlinear equations from the device models, such as $I_D = f(V_{GS}, V_{DS})$, and combines them with the Kirchhoff voltage and current constraints.

These equations are linearised through construction of the Jacobian matrix, which is the multivariable generalisation of the derivative $f'(x)$.

SPICE then iterates using the NewtonRaphson method until all node voltages converge.

The `.OPTIONS` command in SPICE controls convergence parameters such as `RELTOL` (relative tolerance) and `ITL1` (DC iteration limit), which are directly analogous to the tolerance ε used in the Python-based implementation.

2.4 Lab 2

Lab 2: CMOS Inverter Design and Analysis

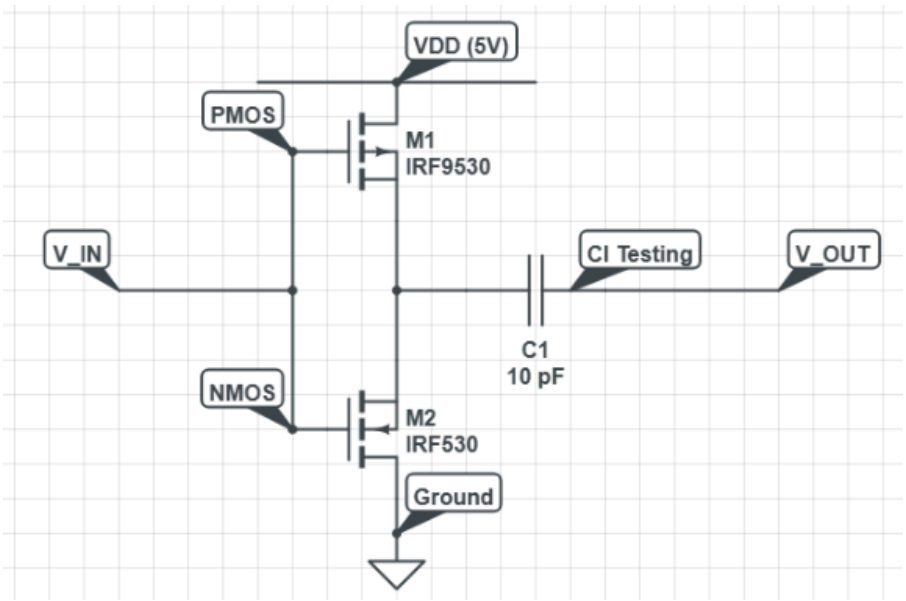

Figure 8: Cobweb Diagram Showing the Convergence of NewtonRaphson Iterations.

PMOS (pull-up network) 20 μm, 1 μm
NMOS (pull-down network) 10 μm, 1 μm
added here as it would not be clear in the image.

SPICE Template Completion

The SPICE netlist defines the circuit topology, device models, and analysis type. For DC analysis, we sweep Vin from 0V to 5V to observe the transfer characteristic.

The netlist code is as follows:

```
* Power supply: 5V DC connected between Vdd node and ground
VDD Vdd 0 DC 5

* Input voltage source: swept from 0 to 5V for DC analysis
Vin in 0 DC 0

* NMOS transistor M1 (pull-down network)
* Terminals: Drain=out, Gate=in, Source=0, Bulk=0
M1 out in 0 0 NMOSLEVEL1 W=10u L=1u

* PMOS transistor M2 (pull-up network)
* Terminals: Drain=out, Gate=in, Source=Vdd, Bulk=Vdd
M2 out in Vdd Vdd PMOSLEVEL1 W=20u L=1u

* DC sweep: Vin from 0V to 5V in 10mV steps
.dc Vin 0 5 0.01

* Plot input and output voltages
.plot DC V(in) V(out)

* NMOS Level 1 model: Vth=0.4V, KP=200uA/V^2, lambda=0.02
.model NMOSLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02

* PMOS Level 1 model: Vth=-0.4V, KP=80uA/V^2, lambda=0.02
.model PMOSLEVEL1 PMOS LEVEL=1 VTO=-0.4 KP=80u LAMBDA=0.02

.end
```

Listing 5: SPICE template completion

The netlist defines complementary NMOS and PMOS transistors that share a common gate (input) and a common drain (output). The PMOS device is sized with a width of $W = 20\,\mu\text{m}$, which is twice that of the NMOS transistor, to compensate for the lower hole mobility. This sizing choice ensures approximately symmetric rise and fall times during switching.

1C: DC Transfer Curve

The DC transfer curve shows the static relationship between input and output. Five operating regions exist based on which transistor is in cutoff, linear, or saturation.

At point A, with $V_{\text{in}} = 0\,\text{V}$, the NMOS transistor has $V_{GS} = 0 < V_{th}$ and is therefore turned off. The PMOS transistor has $V_{SG} = 5\,\text{V} > |V_{th}|$, conducts strongly, and pulls the output voltage such that $V_{\text{out}} \approx V_{DD}$.

At point C, with $V_{\text{in}} = 2.5\,\text{V}$, both transistors operate in saturation. This corresponds to the high-gain transition region, where small changes in the input voltage produce large swings in the output voltage.

At point E, with $V_{\text{in}} = 5\,\text{V}$, the PMOS transistor has $V_{SG} = 0 < |V_{th}|$ and is therefore turned off. The NMOS transistor conducts, pulling the output voltage such that $V_{\text{out}} \approx 0\,\text{V}$.

The transfer curve can be seen clearly below:

Point	V_{in} (V)	V_{out} (V)	NMOS Mode	PMOS Mode
A	0	~ 5	Cutoff	Linear
B	1	~ 5	Saturation	Linear
C	2.5	~ 2.5	Saturation	Saturation
D	4	~ 0	Linear	Saturation
E	5	~ 0	Linear	Cutoff

Table 2: Operating regions of NMOS and PMOS transistors at key points on the CMOS inverter voltage transfer characteristic

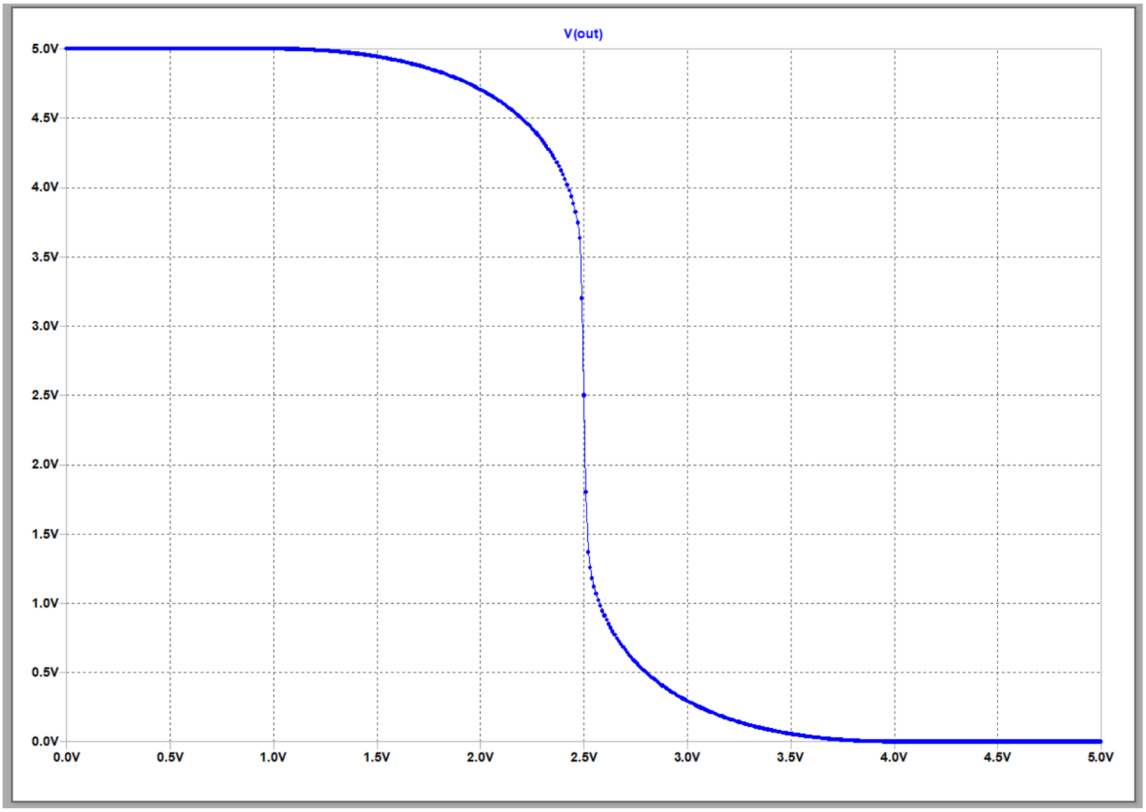

Figure 9: CMOS Transfer Curve

The following figures show the cursor positions for points A through E on the voltage transfer curve.

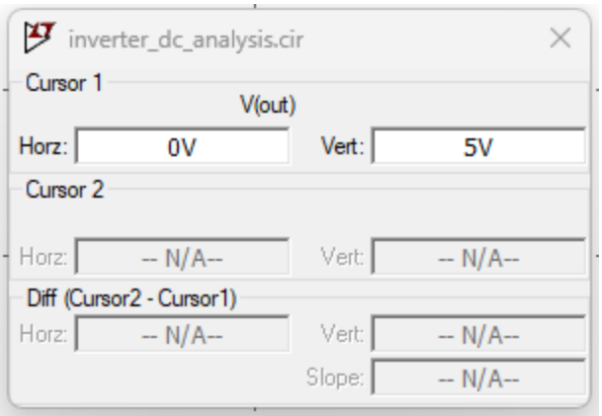

Figure 10: CMOS Point A

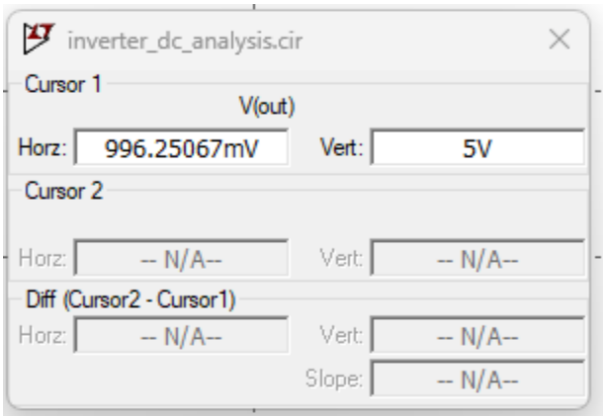

Figure 11: CMOS Point B

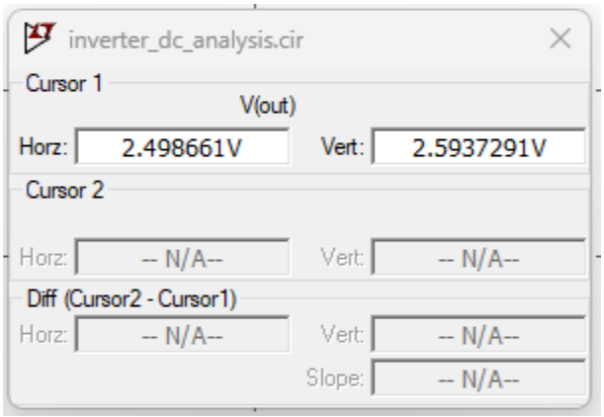

Figure 12: CMOS Point C

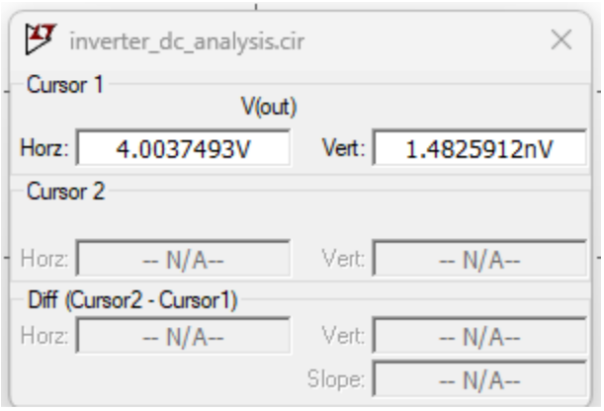

Figure 13: CMOS Point D

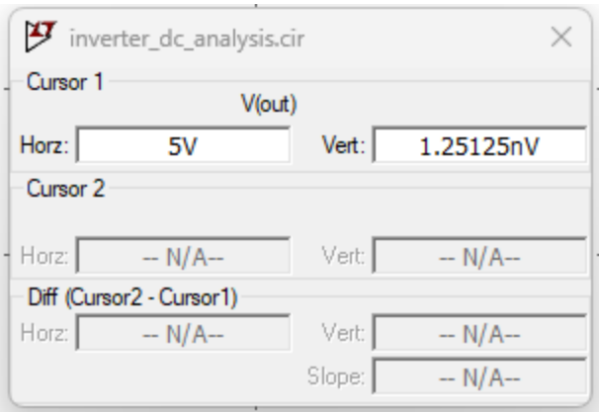

Figure 14: CMOS Point E

1D NMOS Pull Up Exploration

By altering the CMOS structure to only use NMOS transistors for both pull-up and pull-down networks, we can observe the impact on the voltage transfer characteristic. The modified netlist is as follows:

```
* NMOS-only Inverter (Pseudo-NMOS)
VDD Vdd 0 DC 5
Vin in 0 DC 0

* Pull-down NMOS
M1 out in 0 0 NMOSLEVEL1 W=10u L=1u

* Pull-up NMOS (gate tied to VDD - always trying to conduct)
M2 out Vdd Vdd 0 NMOSLEVEL1 W=20u L=1u

.dc Vin 0 5 0.01
.plot DC V(out)
.model NMOSLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02
.end
```

Listing 6: NMOS Pull Down

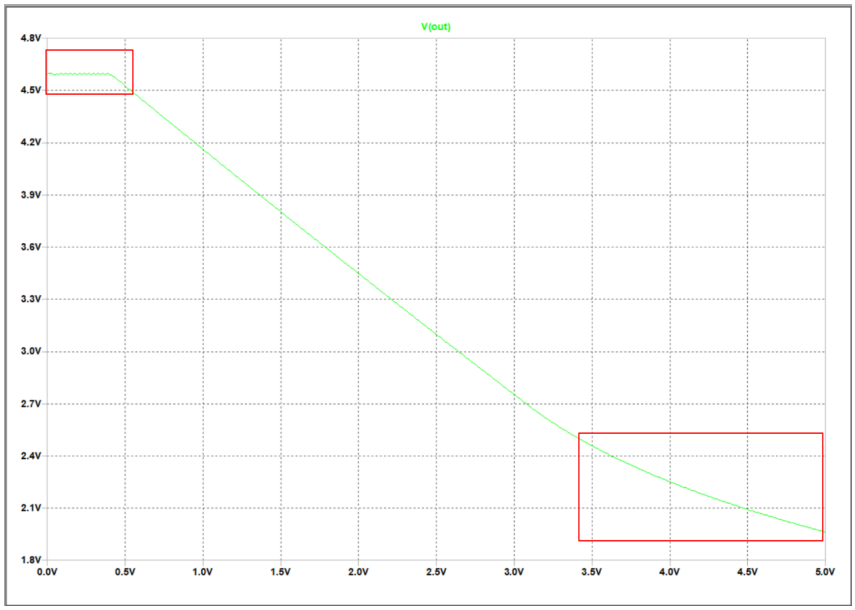

Figure 15: NMOS Pull Down Transfer Curve

Q1: What is the highest output voltage? Why not 5V?

The maximum $Vout \approx 4.6V$ $(= VDD - Vth = 5 - 0.4V)$. The NMOS pull-up cannot pass a voltage higher than $VGS - Vth$. When Vout approaches $VDD - Vth$, the pull-up NMOS enters cutoff because $VGS = VDD - Vout$ drops below Vth.

Q2: What happens to the NMOS pull-up when Vin = 0?

When $Vin = 0V$, the pull-down M1 is OFF. The pull-up M2 tries to charge the output, but it's operating as a source follower with $VGS = VDD - Vout$. As Vout rises, VGS decreases until $VGS = Vth$, at which point M2 enters cutoff and charging stops.

Q3: What happens if this drives another CMOS gate?

The degraded logic '1' ($4.6V$ instead of $5V$) may not fully turn OFF the PMOS in the next stage. This causes:

- Static power dissipation (both transistors partially ON)
- Reduced noise margin
- Slower switching in subsequent stages

Q4: Why does PMOS avoid this issue?

PMOS passes strong '1's because its source connects to VDD. When pulling up, $VSG = VDD - Vout$ remains high until $Vout = VDD$. PMOS can fully charge the output to VDD without the threshold voltage drop limitation.

From the figure above, the appropriate regions have been labelled.

Task 2: Inverter Transient Analysis

Transient analysis characterises the dynamic switching behaviour of the circuit. The key timing parameters are defined as follows.

$$t_{\text{rise}} = t_{90\%} - t_{10\%}, \tag{36}$$

$$t_{\text{fall}} = t_{10\%} - t_{90\%}, \tag{37}$$

$$t_{PHL} = t_{\text{out},50\%} - t_{\text{in},50\%} \quad \text{(output falling)}, \tag{38}$$

$$t_{PLH} = t_{\text{out},50\%} - t_{\text{in},50\%} \quad \text{(output rising)}. \tag{39}$$

The design specifications are:

$$t_{\text{rise}} \leq 15\,\text{ns},$$

$$t_{\text{fall}} \leq 10\,\text{ns},$$

$$|t_{PLH} - t_{PHL}| \leq 2\,\text{ns}.$$

Code and Explanation

```
* CMOS Inverter Transient - Original Design
VDD Vdd 0 DC 5
Vin in 0 PULSE(0 5 0ns 5ns 5ns 50ns 100ns)  * 100ns period pulse

M1 out in 0 0 NMOSLEVEL1 W=10u L=1u
M2 out in Vdd Vdd PMOSLEVEL1 W=20u L=1u
CL out 0 10p                                  * 10pF load

.tran 0.1ns 200ns

.model NMOSLEVEL1 NMOS LEVEL=1 VTO=0.4 KP=200u LAMBDA=0.02
.model PMOSLEVEL1 PMOS LEVEL=1 VTO=-0.4 KP=80u LAMBDA=0.02

* Automated measurements
.meas tran t_rise TRIG V(out) VAL=0.5 RISE=1 TARG V(out) VAL=4.5 RISE=1
.meas tran t_fall TRIG V(out) VAL=4.5 FALL=1 TARG V(out) VAL=0.5 FALL=1
.meas tran tPHL TRIG V(in) VAL=2.5 RISE=1 TARG V(out) VAL=2.5 FALL=1
.meas tran tPLH TRIG V(in) VAL=2.5 FALL=1 TARG V(out) VAL=2.5 RISE=1
.meas tran t_skew PARAM='abs(tPHL-tPLH)'
.end
```

Listing 7: NMOS Pull Down

The `PULSE` source generates a $0 - 5V$ square wave with $5ns$ rise/fall times and $100ns$ period. The `.meas` commands automatically extract timing parameters.

```
LTspice 24.1.6 for Windows
Circuit: C:\Users\sahas\OneDrive\Documents\LTspice\NewLTspice\inverter_transient_with_meas.cir
Start Time: Sun Dec 28 13:36:03 2025
solver = Normal
Maximum thread count: 4
tnom = 27
temp = 27
method = trap
Instance "M1": Length shorter than recommended for a level 1 MOSFET.
Instance "M1": Width narrower than recommended for a level 1 MOSFET.
Instance "M2": Length shorter than recommended for a level 1 MOSFET.
Direct Newton iteration for .op point succeeded.
Total elapsed time: 0.440 seconds.

Files loaded:
C:\Users\sahas\OneDrive\Documents\LTspice\NewLTspice\inverter_transient_with_meas.cir

t_rise=4.31961111041e-09 FROM 5.81766831026e-08 TO 6.24962942131e-08
t_fall=3.72037502918e-09 FROM 2.94452195604e-09 TO 6.66489698522e-09
tphl=2.16419158955e-09 FROM 2.49999995702e-09 TO 4.66419154657e-09
tplh=2.5167216919e-09 FROM 5.74999999869e-08 TO 6.00167216788e-08
t_prop: '(tPHL+tPLH)/2'=2.34045664073e-09
t_skew: 'abs(tPHL-tPLH)'=3.5253010234e-10
```

Figure 16: Original Transient Analysis Log Output

Question 2, RC Model Delay

MOS transistors can be modelled as ideal switches with a finite on-resistance:

$$R_{\text{on}} \approx \frac{V_{DD}}{I_{D,\text{avg}}}$$

When operating in saturation, the drain current is given by:

$$I_{D,\text{sat}} = \frac{K_P}{2}\frac{W}{L}\left(V_{GS} - V_{th}\right)^2$$

The switching behaviour is dominated by the RC time constant formed by the on-resistance and load capacitance:

$$\tau = R_{\text{on}} C_L$$

The rise and fall times are approximately related to the time constant by:

$$t_{\text{rise/fall}} \approx 2.2\tau$$

Results and Discussion

$$I_{D,\text{sat}} = \frac{200\mu}{2} \times 10 \times (5 - 0.4)^2 = 21.16 \text{ mA}$$

$$R_{\text{NMOS}} \approx \frac{5 \text{ V}}{I_{D,\text{sat}}/2} = \frac{5}{21.16 \text{ mA}/2} = 473 \ \Omega$$

$$I_{D,\text{sat}} = \frac{80\mu}{2} \times 20 \times (5 - 0.4)^2 = 16.93 \text{ mA}$$

$$R_{\text{PMOS}} \approx \frac{5\text{ V}}{I_{D,\text{sat}}/2} = \frac{5}{16.93\text{ mA}/2} = 591\ \Omega$$

$$t_{\text{fall}} = 2.2 \times R_{\text{NMOS}} \times C_L = 2.2 \times 473 \times 10\text{ pF} = 10.4\text{ ns}$$

$$t_{\text{rise}} = 2.2 \times R_{\text{PMOS}} \times C_L = 2.2 \times 591 \times 10\text{ pF} = 13.0\text{ ns}$$

In reality, the measured rise and fall times from the simulation are actually much faster, showing that the inverter actually meets the design specifications. This discrepancy arises because the simplified RC model neglects the nonlinear behaviour of the transistors, which can provide lower effective resistance during switching.

This can be seen from the two figures below, which show the annotated waveforms for rise and fall times and the log file.

IMAGE SHOWING THE ANNOTATED WAVEFORM OF RISE AND FALL TIMES HERE.

IMAGE SHOWING THE TRANSIENT ANALYSIS LOG OUTPUT HERE.

Question 3, Design Optimisation and Testing

The original design meets all timing specifications. However, to further optimise performance, we can adjust the transistor sizes. By increasing the NMOS width to $W_{NMOS} = 15\,\mu$m and the PMOS width to $W_{PMOS} = 30\,\mu$m, we can reduce the on-resistances, thereby decreasing rise and fall times. This also allows for symmetric switching characteristics and delays.

The netlist can be seen here: LINK TO GITHUB

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE OPTIMISED RISE AND FALL TIMES HERE.

The log file also shows that the optimised design meets all timing specifications with improved margins (nearly 1ns faster for t_rise).

Asymmetric Ratio Test

Parameter	**Value**	**Change**
W_{NMOS}	15 μm	+50%
W_{PMOS}	37.5 μm	+87.5%
Ratio (W_P/W_N)	2.5	Optimal

Table 3: Variant 2 transistor sizing for faster switching

The same width ratio of 2.5 is retained to maintain symmetric rise and fall times. Increasing transistor widths further reduces the on-resistance, leading to faster switching at the expense of increased silicon area.

$$R_{\text{NMOS}} = \frac{1}{200\mu \times 15 \times 4.6} = 72\ \Omega$$

$$R_{\text{PMOS}} = \frac{1}{80\mu \times 37.5 \times 4.6} = 72\ \Omega$$

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE ASYMMETRIC RISE AND FALL TIMES HERE.

Also, we can see that there is an increase in times (0.8ns for rise and 0.4 ns for fall) due to the larger capacitance associated with wider transistors, which slightly offsets the benefits of reduced resistance.

Further evidence can be seen in the log file below.

IMAGE SHOWING THE TRANSIENT ANALYSIS LOG OUTPUT OF THE ASYMMETRIC TEST HERE.

For further testing, one could explore the output after doubling the capacitance or halving the width, to verify the rules of proportionality.

The output waveforms and log files for these tests can be found here: LINK TO GITHUB

Question 4, Qualitative Analysis

Procedure and Theory The delay of a CMOS inverter is governed by the RC time constant of the conducting transistor and the load capacitance. Hence,

$$t \propto R_{on} \times C_L$$

and since the on-resistance depends inversely on the process transconductance parameter and device width,

$$R_{on} \propto \frac{1}{K_P \cdot W}, \qquad t \propto \frac{C_L}{K_P \cdot W}.$$

This shows that increasing transistor width or carrier mobility reduces delay, while increasing load capacitance increases delay.

Results and Discussion Q4a: Why does the NMOS produce a faster fall time than the PMOS rise time?

Electron mobility μ_n is approximately 2.5 times greater than hole mobility μ_p. Since $K_P = \mu C_{ox}$, this results in

$$K_{P,n} = 200\ \mu\text{A/V}^2, \qquad K_{P,p} = 80\ \mu\text{A/V}^2.$$

The lower K_P of the PMOS leads to a higher on-resistance compared to the NMOS for equal widths. Consequently, charging the load capacitance (rise time) is slower than discharging it (fall time), giving a longer t_{PLH} than t_{PHL} in an unbalanced design.

Q4b: What happens if W_{NMOS} and W_{PMOS} are halved?

Since $R_{on} \propto 1/W$, halving the transistor widths doubles the on-resistance. In theory, this results in approximately doubled delays:

$$t_{rise} :\sim 13\ \text{ns} \rightarrow\sim 26\ \text{ns}, \qquad t_{fall} :\sim 10\ \text{ns} \rightarrow\sim 20\ \text{ns}.$$

These values would violate the timing specification. However, it should be noted that the implemented designs were sized to meet the specification, and this degradation is a theoretical consequence of width reduction rather than an observed failure in the final designs.

Q4c: What happens if the load capacitance doubles to 20 pF?

From $\tau = RC$, doubling C_L directly doubles the delay. In theory,

$$t_{rise} :\sim 13 \text{ ns} \rightarrow\sim 26 \text{ ns}, \qquad t_{fall} :\sim 10 \text{ ns} \rightarrow\sim 21 \text{ ns}.$$

Again, while this illustrates the sensitivity of delay to load capacitance, both the original and optimised designs were verified to meet the specification under the intended loading conditions.

Q4d: Why is symmetric t_{PHL} and t_{PLH} important?

Symmetric rise and fall delays preserve signal duty cycle, which is particularly important for clocked systems. Predictable and balanced delays improve timing margins by ensuring consistent setup and hold times, and prevent cumulative distortion when signals propagate through cascaded logic stages. Additionally, symmetric transitions reduce short-circuit power by minimising overlap between NMOS and PMOS conduction during switching.

FPGA Design Labs (Labs 4, 5 & 6)

3.1 Lab 4, IPSP (Single-Cycle and Pipelined)

Theory and Procedure

The IPSP performs a multiply-accumulate (MAC) operation:

$$y_{out} = a \times x + y_{in} \tag{40}$$

In systolic arrays, IPSPs chain together—each receives a partial sum from its neighbour, adds its contribution, and passes the result onward.

Rather than complex floating-point hardware, this lab uses Q16.16 fixed-point representation, consisting of 16 integer bits and 16 fractional bits stored in a 32-bit word.

When multiplying two Q16.16 numbers, the product is a 64-bit Q32.32 value. To return to Q16.16 format, bits [47:16] of the product are extracted:

$$\text{Result} = \text{Product}[47 : 16] \tag{41}$$

The 64-bit product can be interpreted as:

63 . . . 48	47 . . . 32	31 . . . 16	15 . . . 0
overflow	integer	fraction	discard

A single-cycle design requires that the result appears one clock cycle after valid input. The entire multiply-and-add combinational path must therefore complete within a 10 ns

clock period at 100 MHz. This constraint is challenging because the combinational delay often exceeds the available timing budget.

Results and Discussion

The testbench verifies functional correctness. When `valid_in` is asserted, `valid_out` goes high on the next clock edge with the correct result.

For example, with

$$a = 2.0, \quad x = 3.0, \quad y_{in} = 4.0 \tag{42}$$

the output is

$$y_{out} = 10.0 \tag{43}$$

which corresponds to the Q16.16 hexadecimal representation

$$y_{out} = \texttt{0x000A0000}. \tag{44}$$

Waveforms should be annotated to show the single-cycle input-to-output relationship, and Q16.16 values should be verified by changing the radix from hexadecimal to decimal for clarity.

Synthesis converts HDL code into a technology-mapped netlist by inferring logic such as adders, multipliers, and registers, providing an idealised view of resource usage and timing.

Implementation maps this netlist onto the physical device through placement and routing, introducing real interconnect delays. As a result, implementation determines whether the design meets its clock timing constraints, while synthesis only determines the logical hardware structure.

Firstly, a visual verification of the waveform shows that, after 10ns, the output matches the expected result when valid input is provided, with the rising edge for the input at 35ns and the corresponding output at 45ns.

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE SINGLE CYCLE IPSP HERE.

Worst Negative Slack (WNS) Worst Negative Slack is the minimum setup-time slack over all synchronous timing paths in the design. It indicates how much the most critical path violates (or meets) the clock period constraint:

$$\text{WNS} = \min\left(T_{\text{required}} - T_{\text{arrival}}\right)$$

A negative WNS implies a setup-time violation and limits the maximum achievable clock frequency, while a positive WNS indicates available timing margin.

Worst Pulse Width Slack (WPWS) Worst Pulse Width Slack measures the minimum slack associated with clock pulse width constraints at sequential elements:

$$\text{WPWS} = \min\left(T_{\text{pulse,actual}} - T_{\text{pulse,required}}\right)$$

A negative WPWS indicates that the clock high or low pulse width is too short to meet the flip-flop or latch requirements, potentially causing functional failure independent of clock frequency.

This can be seen in the timing report excerpt below, where an infinite negative WNS indicate that the single-cycle design cannot meet the 10 ns clock period due to excessive combinational delay, thus highlighting the impracticality of this approach for the given operation.

IMAGE SHOWING THE TIMING REPORT OF THE SINGLE CYCLE IPSP HERE.

The Power Summary estimates the on-chip power consumption of an FPGA design. Total power is reported as the sum of dynamic and static components:

$$P_{\text{total}} = P_{\text{dynamic}} + P_{\text{static}}$$

Dynamic power arises from signal switching activity and is broken down by resource type, including clock networks, logic, routing, memory blocks, DSPs, and I/O. Static power represents leakage power and depends primarily on the device, operating voltage, and temperature.

The Power Summary also provides estimated thermal information, such as junction temperature, enabling assessment of the design's power efficiency and thermal feasibility. The reported values are estimates and depend on the accuracy of clock and switching activity assumptions.

IMAGE SHOWING THE POWER REPORT OF THE SINGLE CYCLE IPSP HERE.

For further reference, the utilisation report can simply tell a designer how many resources have been used in the FPGA, such as LUTs, Flip-Flops, BRAMs, and DSP slices.

IMAGE SHOWING THE UTILISATION REPORT OF THE SINGLE CYCLE IPSP HERE.

Synthesis and Implementation of IPSP

After behavioural simulation, the design proceeds through synthesis and implementation, with each stage increasing realism: behavioural simulation assumes ideal zero-delay operation, post-synthesis timing includes estimated delays, and post-implementation timing accounts for actual routing delays and is the most accurate.

Synthesising in out-of-context (OOC) mode analyses a module in isolation, reporting its intrinsic timing characteristics without influence from the parent design.

The Artix-7 FPGA contains DSP48E1 primitives, which are hardwired multiplyaccumulate blocks. Vivado automatically maps 32×32 multiplications to these primitives, enabling fast multiplication, whereas division has no equivalent dedicated hardware.

The timing report identifies the critical path as the slowest path through the design, typically from input, through the DSP48E1 multiplier, then fabric-based addition logic, to the output.

This can be seen in the timing report excerpt below:

IMAGE SHOWING THE TIMING REPORT OF THE SINGLE CYCLE IPSP POST IMP HERE.

Pipelined IPSP, Theory and Procedure

When a single-cycle design fails timing, pipelining divides the long combinational path into shorter stages by inserting registers. The critical path of a pipelined design is therefore

$$T_{\text{critical,pipelined}} = \max(T_{\text{stage1}}, T_{\text{stage2}}, \ldots) < T_{\text{stage1}} + T_{\text{stage2}} + \ldots \tag{45}$$

A four-stage pipeline can be described as follows: 1. stage 1 registers the inputs (a, x, y); 2. stage 2 performs the multiplication; 3. stage 3 registers the product and aligns y; 4. stage 4 performs the addition and produces the output.

Compared to a single-cycle design, pipelining increases latency from one cycle to four cycles while preserving a throughput of one result per cycle. The achievable clock frequency improves from approximately 80 MHz to above 100 MHz. After the initial four-cycle delay, one result is produced every cycle, which is naturally tolerated by systolic arrays.

The `valid` signal must be pipelined alongside the data so that `valid_out` is asserted only when valid data reaches the output stage.

This approach lets more hardware resources be used in parallel, trading off area for speed.

Results and Discussion

Synthesis and implementation proceed similarly to the single-cycle design, with the addition of pipeline registers breaking the critical path.

The waveform also adheres to the expected four-cycle latency, with `valid_out` asserted four cycles after `valid_in`, and the output data correctly reflecting the pipelined multiply-accumulate operation.

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE PIPELINED IPSP HERE.

The timing report excerpt below shows that the pipelined design meets the 10 ns clock period with a positive WNS, indicating successful timing closure.

IMAGE SHOWING THE TIMING REPORT OF THE PIPELINED IPSP POST SYNTHESIS HERE.

After implementation, the timing report confirms that the design still meets timing constraints, with the critical path now slightly shorter due to pipelining.

IMAGE SHOWING THE TIMING REPORT OF THE PIPELINED IPSP POST IMP HERE.

As mentioned above, the critical path can be seen here as well. The critical path tells the designer where the longest delay occurs, which is useful for further optimisation if needed.

IMAGE SHOWING THE CRITICAL PATH OF THE PIPELINED IPSP HERE.

The power reports show the dynamic and static power consumption of the pipelined design, which may be higher than the single-cycle design due to increased register usage but is offset by improved timing performance.

There is a higher utilisation of the FPGA resources due to the additional pipeline registers, which is reflected in the utilisation report.

IMAGE SHOWING THE POWER REPORT OF THE PIPELINED IPSP HERE POST SYNTH.

And after implementation:

IMAGE SHOWING THE POWER REPORT OF THE PIPELINED IPSP HERE POST IMP.

Utilisation report shows the resource usage of the pipelined design, which typically increases due to the additional registers but remains within acceptable limits for the target FPGA.

IMAGE SHOWING THE UTILISATION REPORT OF THE PIPELINED IPSP HERE.

2D, Timing and Understanding

This task requires explaining why pipelining improves performance and identifying the FPGA primitives used in the design. Pipelining helps by splitting a long combinational path into shorter stages separated by registers, reducing the critical path delay and allowing a higher clock frequency.

By adjusting the XDC clock constraints, the maximum operating frequency of each design can be determined. The pipelined design achieves a significantly higher $F_{\max}$ because each pipeline stage contains less combinational logic, whereas the single-cycle design must complete the entire operation within one clock period.

The FPGA primitives used are summarised below. Multiplication is mapped to the DSP48E1 primitive because Artix-7 devices contain dedicated hardwired multiplier–accumulator blocks that provide high-speed arithmetic. Addition is implemented using LUTs with CARRY8 chains, which are optimised for fast carry propagation across the FPGA fabric. Registers are implemented using FDRE primitives, which are edge-triggered flip-flops used for state storage and pipelining.

Operation	**Primitive**
Multiplication	DSP48E1
Addition	CARRY8 + LUTs
Registers	FDRE

3.2 Lab 5, DivSub

Single Cycle DivSub, Theory and Procedure

The `DIVSUB` block is located at the left boundary of the systolic array and computes

$$x_{out} = \frac{b - y}{a}. \tag{46}$$

This operation consists of a subtraction followed by a division. Unlike multiplication, FPGAs provide no dedicated hardware for division, so it must be implemented using LUTs and general fabric logic.

In the first version, both subtraction and division are implemented as purely combinational logic, completing in a single clock cycle. For a 32-bit division, this requires fully unrolling the division algorithm into one large combinational path.

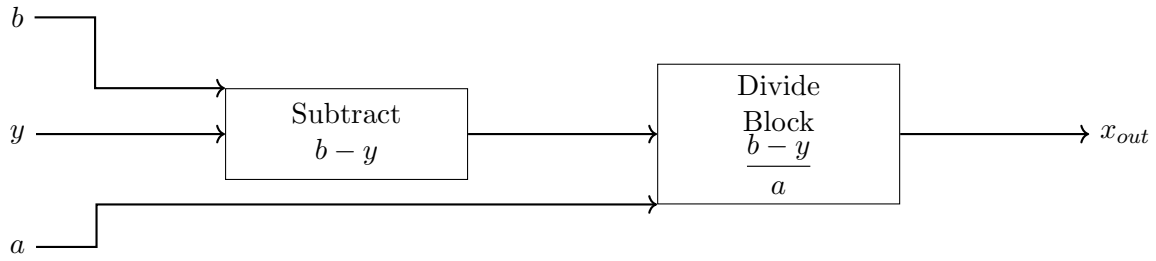

Figure 17: DIVSUB block with right-angled signal routing showing subtraction followed by division

A combinational divider for a WIDTH-bit operand requires WIDTH iterations of compare-subtractshift logic, all cascaded into a single path. As a result, the critical path delay grows proportionally to

$$O(\text{WIDTH}^2) \tag{47}$$

gate delays. For 32-bit operands, this produces thousands of logic levels, far exceeding any practical clock period and causing severe timing failures.

For fixed-point Q16.16 division, direct division causes the fractional scaling to be lost:

$$\frac{A_{Q16.16}}{B_{Q16.16}} = \frac{A \times 2^{16}}{B \times 2^{16}} = \frac{A}{B}, \tag{48}$$

which is no longer in Q16.16 format. To preserve the fixed-point representation, the dividend must be pre-shifted left by 16 bits:

$$\text{Result}_{Q16.16} = \frac{A_{Q16.16} \ll 16}{B_{Q16.16}}. \tag{49}$$

Results and Discussion

The combinational DIVSUB produces functionally correct results in behavioural simulation. Since there are no delays modelled, the division completes instantaneously within the simulation timestep.

However, synthesis and implementation reports reveal that the design cannot meet timing constraints due to the excessively long combinational path through the divider. The timing report shows an infinite WNS, indicating that the critical path delay far exceeds the 10 ns clock period.

The following figures show the waveform, post-synthesis timing report, and post-implementation timing report for the combinational DIVSUB design, along with the equivalent power reports

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE COMBINATIONAL DIVSUB HERE.

IMAGE SHOWING THE TIMING REPORT OF THE COMBINATIONAL DIVSUB POST SYNTH HERE.

IMAGE SHOWING THE TIMING REPORT OF THE COMBINATIONAL DIVSUB POST IMP HERE.

IMAGE SHOWING THE POWER REPORT OF THE COMBINATIONAL DIVSUB HERE POST SYNTH.

IMAGE SHOWING THE POWER REPORT OF THE COMBINATIONAL DIVSUB HERE POST IMP.

Pipelined DivSub, Theory and Procedure

Version 2 implements division sequentially over `WIDTH` clock cycles using the shift–subtract algorithm, which is a practical approach for FPGA implementation.

Each clock cycle computes one bit of the quotient. The remainder register is shifted left by one bit, and the most significant bit of the dividend is copied into the least significant bit of the remainder. The remainder is then compared with the divisor. If

$$\text{Remainder} \geq \text{Divisor}, \tag{50}$$

the divisor is subtracted from the remainder and the next quotient bit is set to 1. If

$$\text{Remainder} < \text{Divisor}, \tag{51}$$

no subtraction occurs and the quotient bit is set to 0. The quotient register is shifted left and the new bit inserted, while the dividend is also shifted left, discarding the bit just used. After `WIDTH` iterations, the quotient register holds the final result.

An example of this process for $13 \div 3$ using 8-bit registers is shown in Table 4.

Cycle	Remainder	Dividend	Quotient	Action
Init	00000000	00001101	00000000	Load
1	00000000	00011010	00000000	$0 < 3$, Q=0
2	00000000	00110100	00000000	$0 < 3$, Q=0
3	00000001	01101000	00000000	$1 < 3$, Q=0
4	00000011	11010000	00000000	$3 \geq 3$, Q=1
5	00000001	10100000	00000001	Subtract, $1 < 3$
6	00000010	01000000	00000010	$2 < 3$, Q=0
7	00000101	10000000	00000100	$5 \geq 3$, Q=1
8	00000001	00000000	00000100	Done: 4 r 1

Table 4: Serial binary division of $13 \div 3$ using an 8-bit shift–subtract algorithm

IMAGE SHOWING THE DIVSUB BLOCK DIAGRAM WITH THE SERIAL DIVIDER HERE.

fp_sub: Computes $(b - y)$ in one cycle

Multi-stage Divider: Sequential division over WIDTH cycles

FIFO Buffer: Stores incoming data while division completes

FSM Control: Orchestrates dataflow and valid signals

New data may arrive every clock cycle, but division takes 32+ cycles. The FIFO buffers incoming operands so they aren't lost while a division is in progress.

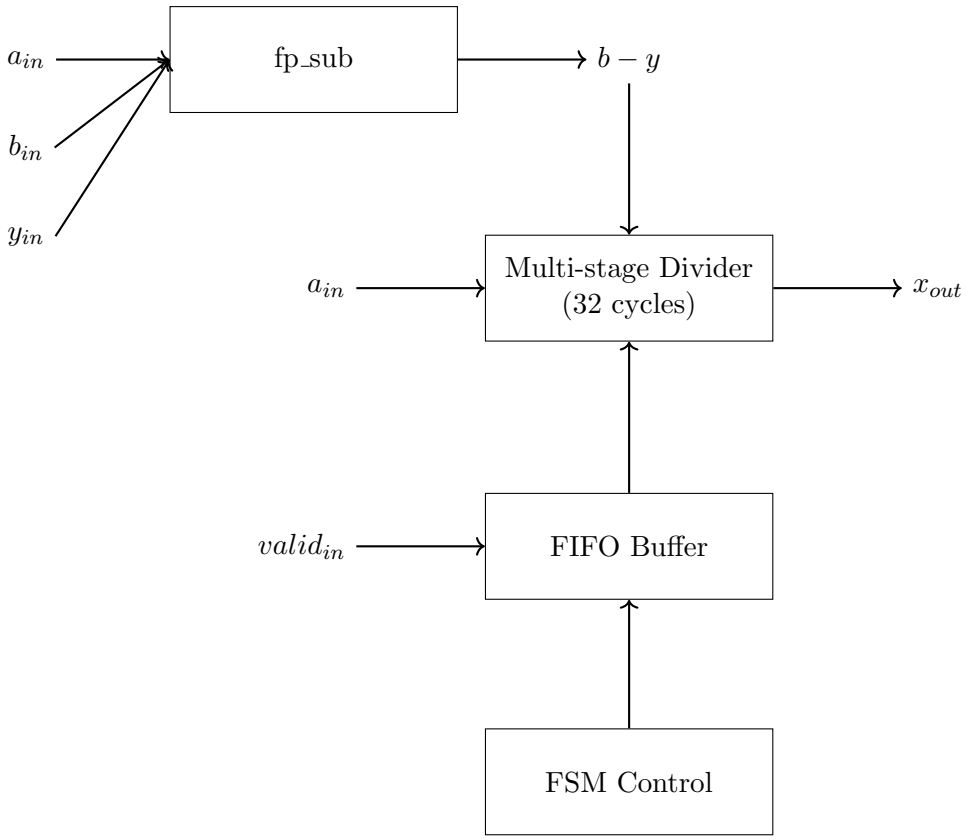

Figure 18: DIVSUB block with pipelined subtraction, multi-cycle division, and control logic. The subtraction computes $b - y$, which is divided by a_{in} using a 32cycle sequential divider. Valid signals are buffered and controlled by an FSM.

Results and Discussion

Subtraction Phase (Cycle 1):

$valid_in$ asserts with inputs
$(b - y)$ computed immediately
FSM transitions from IDLE to DIVIDING

Division Phase (Cycles 233):

Internal counter increments each cycle (visible if exposed)
Remainder and quotient registers update progressively
FSM remains in DIVIDING state

Output Phase (Cycle 34):

Division complete, $valid_out$ asserts
x_out contains correct quotient
FSM returns to IDLE (or processes next FIFO entry)

This can be seen in the waveform below, where $valid_out$ goes high 34 cycles after $valid_in$, and x_out holds the correct result.

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE PIPELINED DIVSUB HERE.

For further reference, the synthesis and implementation timing reports confirm that the pipelined design meets timing constraints, with a positive WNS indicating successful timing closure.

IMAGE SHOWING THE TIMING REPORT OF THE PIPELINED DIVSUB POST SYNTH HERE.

IMAGE SHOWING THE TIMING REPORT OF THE PIPELINED DIVSUB POST IMP HERE.

The power reports show the dynamic and static power consumption of the pipelined DIVSUB design, which may be higher than the combinational design due to increased register usage but is offset by improved timing performance.

IMAGE SHOWING THE POWER REPORT OF THE PIPELINED DIVSUB HERE POST SYNTH.

IMAGE SHOWING THE POWER REPORT OF THE PIPELINED DIVSUB HERE POST IMP.

Register	**Width**	**Purpose**
Dividend	32 bits	Holds the numerator and shifts left each cycle
Divisor	32 bits	Holds the denominator and remains constant during division
Remainder	33 bits	Accumulates partial results during the shift–subtract process
Quotient	32 bits	Builds the final result one bit at a time
Counter	5–6 bits	Tracks division progress from cycle 0 to 31

Table 5: Purpose and width of key registers used in the sequential divider

Timing and Understanding

The tables below compare multiplication and division on FPGAs, highlighting the fundamental differences in their hardware implementations and performance characteristics.

Aspect	**Multiplication**	**Division**
FPGA Primitive	DSP48E1 (dedicated)	None
Algorithm	Parallel array	Sequential shift–subtract
Single-cycle operation	Yes (via DSP)	No (excessive logic depth)
Typical latency	1–4 cycles	`WIDTH` cycles

Table 6: Comparison of multiplication and division hardware on an FPGA

Multiplication has a regular and highly parallelisable structure, allowing partial products to be computed and summed simultaneously. This maps efficiently to dedicated DSP48E1 hardware blocks. In contrast, division is inherently sequential, as each quotient bit depends on the previous remainder, making it unsuitable for single-cycle implementation in general-purpose logic.

This asymmetry explains why systolic arrays place the `DIVSUB` block at the array boundary, where fewer elements are processed, while IPSP units populate the main array to maximise throughput.

Metric	IPSP (Pipelined)	DIVSUB (Multi-cycle)
Operation	$a \times x + y$	$\frac{b-y}{a}$
Latency	4 cycles	$\sim$33 cycles
Throughput	1 per cycle	1 per $\sim$33 cycles
Hardware	DSP48E1	LUTs only

Table 7: Comparison of IPSP and DIVSUB implementations

This was shown in the timing reports for both designs, where the pipelined IPSP meets timing with a positive WNS, while the multi-cycle DIVSUB also meets timing due to its sequential nature, albeit with higher latency.

- No dedicated division hardware: Unlike DSP48E1 for multiplication, FPGAs have no division primitive
- Sequential algorithm required: Division inherently requires iterating through bits — this cannot be parallelised like multiplication
- Multi-cycle is the only practical approach: The choice is between impossibly slow single-cycle or practical multi-cycle
- Latency vs throughput trade-off: Multi-cycle has high latency but reasonable throughput with pipelining/buffering
- System-level impact: The controller (Lab 6) must account for DIVSUB's long latency when scheduling data flow

3.3 Task 6, Full Systolic Array Implementation

The term *systolic* is inspired by the rhythmic pumping of the heart. In a systolic array, data flows through processing elements in a regular, clocked manner. Each processing element performs a simple local operation and passes intermediate results to its neighbours, while all elements operate in lockstep under a global clock. Complex computations therefore emerge from the coordinated operation of many simple units.

Systolic arrays are well suited to solving linear systems of the form

$$Ax = b, \tag{52}$$

particularly when A is triangular and back-substitution can be applied. For a lower triangular system,

$$\begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}, \tag{53}$$

the solution is computed sequentially.

The first variable is obtained directly using a division:

$$x_1 = \frac{b_1}{a_{11}}, \tag{54}$$

which corresponds to a `DIVSUB` operation at the boundary of the array.

The second variable reuses the previously computed result. An IPSP multiplies and accumulates the partial product $a_{21}x_1$, and the result is then divided:

$$x_2 = \frac{b_2 - a_{21}x_1}{a_{22}}. \tag{55}$$

Similarly, the third variable is computed by accumulating multiple partial products across IPSPs before a final division:

$$x_3 = \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}}. \tag{56}$$

As an example, consider the system

$$a_{11} = 2,\ a_{21} = 1,\ a_{22} = 3,\ a_{31} = 1,\ a_{32} = 2,\ a_{33} = 4,\ b_1 = 8,\ b_2 = 13,\ b_3 = 20.$$

The systolic array computes

$$x_1 = \frac{8}{2} = 4,$$

then

$$x_2 = \frac{13 - 1 \cdot 4}{3} = 3,$$

and finally

$$x_3 = \frac{20 - 1 \cdot 4 - 2 \cdot 3}{4} = 2.5.$$

In this way, IPSPs form the main array to perform high-throughput multiply–accumulate operations, while the multi-cycle `DIVSUB` blocks handle division at the array boundary, where fewer operations are required.

- A coefficients: Left-to-right along top
- x values: Left-to-right between cells (solution vector)
- y values: Right-to-left along bottom (partial sums)
- b vector: Enters DIVSUB from below

Results and Discussion

The behavioural simulation waveform confirms that the systolic array correctly computes the solution vector x over multiple clock cycles, with each IPSP and DIVSUB operating in synchrony.

The synthesis and implementation timing reports indicate that the full systolic array meets timing constraints, with a positive WNS demonstrating successful timing closure across the entire design.

IMAGE SHOWING THE ANNOTATED WAVEFORM OF THE FULL SYSTOLIC ARRAY HERE.

IMAGE SHOWING THE TIMING REPORT OF THE FULL SYSTOLIC ARRAY POST SYNTH HERE.

IMAGE SHOWING THE TIMING REPORT OF THE FULL SYSTOLIC ARRAY POST IMP HERE.

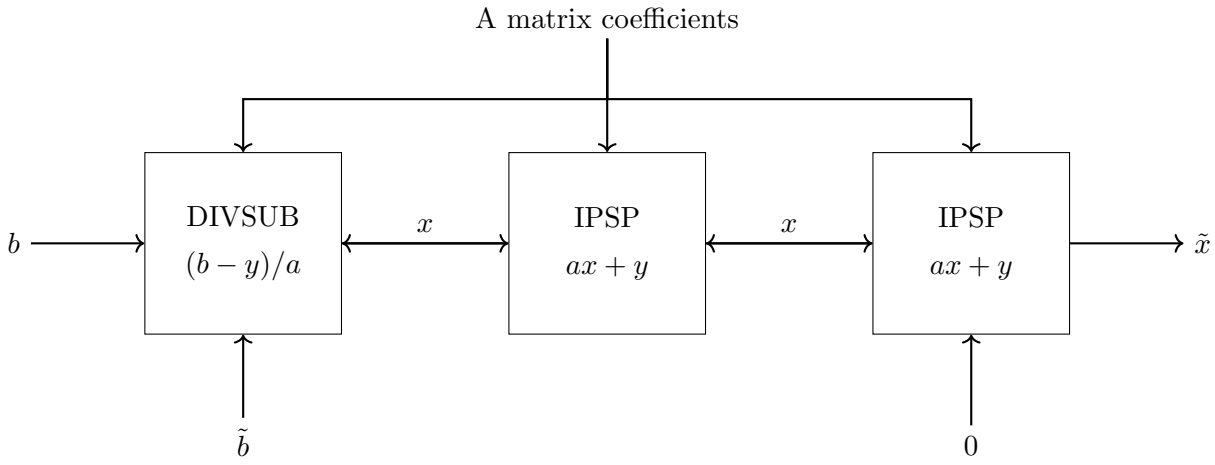

Figure 19: Full systolic array architecture showing the DIVSUB boundary element and chained IPSPs. All data and coefficients propagate synchronously using registered inputs.

Table 8: Key Inter-Block Signals in the Systolic Architecture

Signal	**Source**	**Destination**	**Purpose**
`divsub_x_out`	DIVSUB.`x_out`	IPSP.`x_in`	Propagates the computed solution value into the multiply–accumulate stage
`divsub_valid_out`	DIVSUB.`valid_out`	Controller	Indicates completion of the multi-cycle division operation
`ipsp_y_out`	IPSP.`y_out`	Output / Feedback Path	Carries the accumulated result for output or feedback into subsequent iterations
Controller control signals	Controller	DIVSUB and IPSP blocks	Coordinates operation sequencing and enforces systolic timing

IMAGE SHOWING THE POWER REPORT OF THE FULL SYSTOLIC ARRAY HERE POST SYNTH.

IMAGE SHOWING THE POWER REPORT OF THE FULL SYSTOLIC ARRAY HERE POST IMP.

IMAGE SHOWING THE UTILISATION REPORT OF THE FULL SYSTOLIC ARRAY HERE.

IMAGE SHOWING THE CRITICAL PATH OF THE FULL SYSTOLIC ARRAY HERE.

Asynchronous Circuit Modelling with Workcraft (Labs 7 & 8)

4.1 Lab 7 - C-Element Modelling and Design

Part 1, Theory and Procedure

The C-element is one of the most fundamental building blocks in asynchronous circuit design. Unlike combinational gates where outputs are purely a function of current inputs, the C-element is a state-holding element with memory.

Its behaviour is defined as follows: the output transitions HIGH (Q^+) only when all inputs are HIGH, the output transitions LOW (Q^-) only when all inputs are LOW, and when the inputs have different values, the output retains its previous value. This makes it a synchronisation element that waits for all inputs to agree before changing state.

The state-dependent truth table for a two-input C-element is given by:

A	B	$Q(t)$	$Q(t+1)$
0	0	0	0
0	0	1	0
1	1	0	1
1	1	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1

Signal Transition Graphs (STGs) are an extension of Petri nets specifically designed for specifying asynchronous circuits.

Each signal event (rising transition denoted '+' or falling transition denoted '-') becomes a Petri net transition.

Places between transitions hold tokens representing the current state, and arcs define causality - which events must occur before others can fire.

part 1, C-Element STG construction

Part 1, Results and Discussion

The structure enforces AND-causality by requiring both $in1^+$ and $in2^+$ to fire before out^+ is enabled.

In Petri net terms, out^+ has two input places that must both contain tokens. The implicit places represent waiting states; for example, after $in1^+$ fires, the system waits for $in2^+$ before allowing out^+ to occur.

The initial marking, with tokens on the arcs from out^- to the input rising transitions, represents the starting condition where all signals are LOW and the system is ready for a new SET phase.

The STG allows concurrency, so $in1^+$ and $in2^+$ may occur in any order, reflecting that their relative timing is actually unimportant.

This can be seen in the STG diagram below:

IMAGE SHOWING STG OF C-ELEMENT HERE

The trace corresponding to the STG is shown here, with two runs of the C-element behaviour:

IMAGE SHOWING THE TRACE OF THE C-ELEMENT HERE.

. Circuit timing diagram generated from the trace is shown below:

IMAGE SHOWING THE TIMING DIAGRAM OF THE C-ELEMENT HERE.

Part 2, Simulation and Verification

Part 2, Procedure and theory

Before synthesising an STG into a circuit, it must be verified against key correctness properties.

Consistency requires every signal to alternate strictly between rising and falling transitions, meaning $in1^+ \rightarrow in1^-$ must occur before another $in1^+$, ensuring valid digital behaviour.

Deadlock-freeness guarantees progress by requiring that from every reachable state, at least one transition is enabled, so the system never reaches a state where no action is possible.

Input-properness ensures that input transitions are controlled only by the environment and are never disabled by internal circuit behaviour. Output-persistence requires that once an output or internal transition becomes enabled, it cannot be disabled by another firing transition, which is essential for hazard-free operation.

Verification is performed by first activating the Simulation tool [M] and manually firing enabled transitions, or by pasting a trace such as $in2^+ \rightarrow in1^+ \rightarrow out^+ \rightarrow in1^- \rightarrow in2^- \rightarrow out^-$.

After simulating a trace, a timing diagram can be generated using the *Generate trace diagram* option.

Formal verification is then carried out via *Verification → Consistency (MPSat)*, followed by deadlock freeness, input properness, and output persistency checks, or by using the combined option *Conformation, deadlock freeness, and output persistency (reuse unfolding).*

Part 2, Results and Discussion

Consistency holds because the STG enforces strict alternation of transitions, with each signal following a cyclic pattern $+ \rightarrow - \rightarrow + \rightarrow -$.

Deadlock-freeness is satisfied since at least one transition is always enabled: initially $in1^+$ and $in2^+$ are enabled, after they fire out^+ becomes enabled, and the cycle repeats indefinitely.

Input-properness holds because input transitions ($in1^{\pm}$, $in2^{\pm}$) are controlled solely by the environment and are not disabled by output behaviour.

Output-persistence is satisfied because once out^{+} or out^{-} becomes enabled, it cannot be disabled by any input transition; for example, after $in1^{+}$ and $in2^{+}$ fire, out^{+} is guaranteed to occur since no input can remove the required tokens.

Verification results from Workcraft are shown below:

IMAGE SHOWING THE VERIFICATION RESULTS FOR C-ELEMENT HERE.

Part 3, Synthesis - Complex Gate vs Technology Mapping

Part 3, Procedure and Theory

Asynchronous circuit synthesis converts an STG specification into a gate-level implementation. Workcraft supports two main approaches. In complex-gate synthesis, a single Boolean equation is derived for each output or internal signal. For the C-element this gives

$$out = (out \vee in2) \wedge in1 \ \vee \ in2 \wedge out,$$

which simplifies to

$$out = (in1 \wedge in2) \ \vee \ (in1 \wedge out) \ \vee \ (in2 \wedge out).$$

This equation represents the behaviour of the C-element as a single complex Boolean function and is not constrained to a specific standard cell library.

In technology mapping synthesis, the Boolean equations are mapped onto gates available in a standard cell library, such as NAND, NOR, AND, OR, and inverters. Depending on the target library, the function may be implemented using one gate or decomposed into multiple gates, producing a circuit that is directly implementable in conventional digital technologies.

The synthesis procedure consists of opening the verified C-element STG, running *Synthesis → Complex gate (MPSat)* to generate and record the result, then returning to the STG and running *Synthesis → Technology mapping (MPSat)* or *Petrify* to obtain and record the mapped implementation.

Part 3, Results and Discussion

In this case, technology mapping produces a single gate rather than multiple gates. This occurs because the Boolean function of the C-element can be mapped directly onto a single cell available in the target library. The resulting gate has inputs $in1$ and $in2$ and includes a feedback connection from out to itself, preserving the state-holding behaviour.

The complex-gate synthesis also represents the logic as one combined Boolean function,

$$out = (out \vee in2) \wedge in1 \ \vee \ in2 \wedge out,$$

which is reflected in the Verilog export as a single `assign` statement. In both synthesis approaches, the functional behaviour is identical, and the feedback path ensures correct C-element operation.

For further reference, the Verilog code generated from complex-gate synthesis is shown below:

IMAGE SHOWING THE VERILOG CODE FOR C-ELEMENT HERE.

Workcraft provides two forms of C-element synthesis. Standard C-element synthesis produces a canonical implementation that directly represents an ideal C-element, with built-in memory and feedback. The result typically appears as a single C-element gate or an equivalent standard-cell representation, emphasising clarity and correspondence with theoretical asynchronous models.

Generalised C-element synthesis derives a Boolean function with memory from the STG without enforcing a strict C-element structure. This allows additional logic or asymmetry when required by the specification and may be realised as a complex gate with feedback or as a small network of standard gates after technology mapping. While both approaches implement equivalent behaviour, the standard method highlights the textbook C-element, whereas the generalised method provides greater flexibility for optimisation and technology mapping.

This can be seen in the images below:

IMAGE SHOWING THE STANDARD C ELEMENT

and for Generalised C-element:

IMAGE SHOWING THE GENERALISED C ELEMENT

Part 4, Gate Level Simulation and Verification (Theory and Procedure)

The complex-gate equation of the C-element can be algebraically rewritten as

$$out = (out \vee in2) \wedge in1 \ \vee \ in2 \wedge out = (in1 \wedge in2) \ \vee \ (in1 \wedge out) \ \vee \ (in2 \wedge out).$$

This form reveals an implementation consisting of three AND gates feeding a single OR gate. The term $in1 \wedge in2$ represents the SET condition, while $in1 \wedge out$ and $in2 \wedge out$ implement the state-holding behaviour when the output is already high. The OR gate combines these terms so that the output is asserted either when both inputs are high or when it must be held high.

In Workcraft, the manual decomposition is performed by creating a new Digital Circuit model and generating three two-input AND gates and one three-input OR gate using the function generator. Input ports $in1$ and $in2$ and an output port out are added using the port generator. The inputs are connected to form the $in1 \cdot in2$, $in1 \cdot out$, and $in2 \cdot out$ terms, the AND outputs are wired to the OR gate, and the OR output is connected to out with feedback paths to the AND gates.

Results and Discussion

This decomposition captures C-element behaviour by combining conditional setting with feedback-based state holding. When both inputs are LOW, all AND gates output zero and the OR gate drives $out = 0$. A single rising input does not change the output, since the set term $in1 \wedge in2$ is false and the feedback terms are inactive. When both inputs are HIGH, the set term becomes true and drives the OR gate high, setting $out = 1$.

Once *out* is high, the feedback terms $in1 \wedge out$ and $in2 \wedge out$ maintain the output if either input remains HIGH. The output only resets when both inputs fall, at which point all AND terms are zero. The feedback paths provide the required memory behaviour.

Compared to a complex-gate implementation, the same logic is realised using four standard gates. This increases fan-out at *out*, which feeds both the external output and feedback paths, and increases fan-in at the OR gate, which combines three signals. This may increase delay and area but improves portability across standard cell libraries.

The following images show the gate-level implementations of the C-element:

IMAGE SHOWING THE GATE LEVEL IMPLEMENTATION OF C-ELEMENT HERE.

IMAGE SHOWING THE STG OF THE GATE LEVEL C-ELEMENT HERE.

TRACE OF THE GATE LEVEL C-ELEMENT HERE.

Part 5, Verification and Understanding

Any circuit implementation must be verified to ensure correct behaviour and freedom from hazards. Consistency ensures that signal transitions alternate correctly, while deadlock-freeness guarantees that the circuit can always make progress. Output persistency is essential for glitch-free operation, ensuring that once an output transition is enabled it cannot be disabled, which also implies hazard-freeness.

For verification, Workcraft internally converts the circuit into an equivalent STG capturing all possible behaviours and checks these properties on the derived model. The verification procedure consists of opening the decomposed circuit from Task-4 and running *Verification → Output persistency (MPSat)* and *Deadlock freeness (MPSat)*, or using the combined option *Conformation, deadlock freeness, and output persistency.*

Results and Discussion

When the environment STG is not attached, verification fails for output persistency. The tool reports a violation trace such as $in2^+ \rightarrow in1^+$, indicating that the output *out* is non-persistent. In this situation, after both input transitions have occurred, the output transition out^+ becomes enabled. However, without environment constraints, the verifier allows input transitions such as $in1^-$ or $in2^-$ to occur immediately. If an input falls before out^+ fires, the enabling condition is removed and the output transition is disabled, resulting in a persistence violation.

This failure does not indicate an implementation error. In a realistic system, the environment does not arbitrarily withdraw inputs, and the original STG specification enforces valid input behaviour. The violation therefore arises from analysing the circuit in isolation rather than from incorrect C-element functionality.

the verification results from Workcraft are shown below, including the violation as well as successful verification when the environment is included:

IMAGE SHOWING THE VERIFICATION RESULTS FOR GATE LEVEL C-ELEMENT HERE.

Part 6, Adding Environments and Their Impact

Procedure and Theory

The verification failure observed previously occurs because the circuit is analysed in isolation, allowing inputs to change arbitrarily at any time. In practice, asynchronous circuits operate within well-defined environments that follow specific interaction protocols. The original STG specification therefore encodes both the circuit behaviour, describing how outputs respond to inputs, and the environment assumptions, constraining when input transitions may occur.

By attaching the STG as an environment, the verifier is restricted to consider only input sequences permitted by the specification. To evaluate this effect, two copies of the circuit are created. In the first, the environment property is set to the original *celement.stg.work* file, while in the second the environment is left unspecified. Output persistency verification is then performed on both versions, demonstrating the impact of environment constraints on correctness checking.

Part 6, Results and Discussion

Verification of the circuit with and without an environment demonstrates the importance of environment constraints. When the environment STG is attached, output persistency verification succeeds, showing that *out* behaves correctly under all allowed input sequences. In contrast, without an environment, verification fails with a trace such as $in2^+ \rightarrow in1^+$, where *out* could be disabled by $in1^-$ or $in2^-$.

This occurs because, in isolation, the verifier assumes inputs can change at any time. After $in2^+$ and $in1^+$ enable out^+, the verifier considers the possibility that an input falls immediately, causing a persistence violation. With the environment attached, the STG constrains input sequences: after $in1^+$ and $in2^+$, only out^+ is enabled, and $in1^-$ or $in2^-$ cannot occur until out^+ fires.

The key insight is that the environment STG serves as a contract: the circuit is guaranteed to behave correctly as long as the environment respects the specified input constraints, and verification confirms compliance with this contract.

IMAGE SHOWING THE VERIFICATION RESULTS FOR C-ELEMENT WITHOUT ENVIRONMENT HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR C-ELEMENT WITH ENVIRONMENT HERE.

Bonus Task

The circuit without an attached environment can be converted to a signal transition graph (STG) to compare its behaviour with the original C-element STG.

The extracted STG is generally more complex than the original because it captures all possible behaviours of the circuit, rather than a single controlled protocol.

While the original STG specifies one particular sequence of inputs and outputs following the C-element handshake, the extracted STG represents every sequence of inputs the circuit can respond to.

This results in additional interleavings and states, showing what the circuit can do rather than what it should do. Consequently, the original STG is effectively a subset of the

extracted STG's behaviours.

IMAGE SHOWING THE EXTRACTED STG FROM THE CIRCUIT HERE.

IMAGE SHOWING THE COMPARISON BETWEEN THE ORIGINAL AND EXTRACTED STG HERE.

The extracted STG includes "extra" behaviours that the original specification doesn't allow, which is why environment constraints are needed.

Part 7, External Circuit Verification

Procedure and Theory

This task evaluates the isochronic fork assumption, a key principle in speed-independent asynchronous design. An isochronic fork assumes that when a signal branches to multiple destinations, the delay difference between the branches is negligible relative to gate delays, so all destinations observe the transition effectively simultaneously.

Two circuits are provided for comparison: a correctly implemented decomposed C-element that assumes isochronic forks, and a modified version in which a buffer is inserted on one branch of a fork, introducing a significant delay and violating the isochronic fork assumption.

For each circuit, verification is performed by attaching the original *celement.stg.work* as the environment and running *Verification* $\rightarrow$ *Output persistency (MPSat)* and the conformation check. The results illustrate the impact of violating the isochronic fork assumption on correctness and hazard freedom.

Results and Discussion

For the circuit without the buffer, all verification checks pass, confirming correct C-element behaviour under the isochronic fork assumption. The circuit therefore operates correctly and is free from hazards. In contrast, the circuit with an inserted buffer fails output persistency verification, producing a violation trace that indicates a hazard.

The failure occurs because the buffer breaks the isochronic fork assumption in the feedback path from *out*. In the decomposed C-element, *out* feeds back to multiple AND gates. Under the isochronic assumption, all feedback paths observe transitions simultaneously.

Introducing a buffer causes one AND gate to observe the change later than the other, creating a temporary inconsistent state.

For example, when both inputs and the output are initially HIGH, dropping one input causes some AND terms to deactivate immediately while another remains active.

If the second input then falls before the delayed feedback is observed, the output may be incorrectly disabled or exhibit non-persistent behaviour. The violation trace reported by Workcraft highlights this window where an enabled output can be disabled.

The key insight is that speed-independent design relies on isochronic forks. While wire delays are usually negligible compared to gate delays, significant interconnect delays or poor routing can violate this assumption. The buffered circuit explicitly models such a

violation, demonstrating why physical implementation considerations remain important in asynchronous circuit design.

Here, the verification results for both circuits are shown:

IMAGE SHOWING THE VERIFICATION RESULTS FOR C-ELEMENT WITH ISOCHRONIC FORK HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR C-ELEMENT WITH BUFFERED FORK HERE.

And the verification violation trace for the buffered fork circuit is shown below:

IMAGE SHOWING THE VERIFICATION VIOLATION TRACE FOR C-ELEMENT WITH BUFFERED FORK HERE.

4.2 Lab 8 - VME Bus Controller

Task 1: Model Read and Write Controllers, Procedure and Theory

The VME (Versa Module Europa) bus is a standard computer bus interface in which a controller mediates communication between the system bus and a slave device. The controller supports two operations: read and write, which differ primarily in the timing of the data transceiver.

READ operation (Device → Bus):

The bus initiates a read using `dsr+`. The controller forwards the request to the device with `lds+`. Once the device has data ready, it responds with `ldtack+`. The controller then enables the data transceiver using `d+` and signals data availability to the bus with `dtack+`. After the transfer completes, all signals return low in sequence:

`dsr-`, `lds-`, `d-`, `ldtack-`, `dtack-`.

WRITE operation (Bus → Device):

The bus requests a write using `dsw+`. The controller first enables the data transceiver with `d+` so that data is present on the bus, then asserts `lds+` to notify the device. The device acknowledges receipt using `ldtack+`. The controller then disables the transceiver with `d-` and signals completion to the bus using `dtack+`. The remaining signals return low in order:

`dsw-`, `lds-`, `ldtack-`, `dtack-`.

The key difference between the two modes is the timing of the transceiver control. In the read operation, the transceiver is enabled only after the device confirms data readiness, whereas in the write operation it is enabled before the write request to ensure data is available to the device.

Inputs from the environment are `dsr`, `dsw`, and `ldtack`. Outputs generated by the controller are `lds`, `d`, and `dtack`.

For the read controller, a new STG was created in Workcraft and saved as `vme-read.stg.work`.

Transitions were added following the read timing sequence, signal directions were assigned appropriately, and causal arcs were used to enforce ordering.

An initial token was placed before `dsr+`.

For the write controller, a separate STG was created and saved as `vme-write.stg.work`.

The write timing sequence was implemented, with particular attention to the earlier assertion of `d+`. Signal types were assigned and an initial token was placed to represent the idle state.

Results and Discussion

Figures of the READ and WRITE STGs are included. Both models exhibit largely linear behaviour, with limited concurrency during the reset phase.

In the **READ STG**, the sequence

$$\texttt{dsr+} \rightarrow \texttt{lds+} \rightarrow \texttt{ldtack+} \rightarrow \texttt{d+} \rightarrow \texttt{dtack+}$$

ensures that the device signals data readiness before the transceiver is enabled. This ordering prevents invalid or unstable data from propagating onto the bus.

In the **WRITE STG**, the sequence

$$\texttt{dsw+} \rightarrow \texttt{d+} \rightarrow \texttt{lds+} \rightarrow \texttt{ldtack+} \rightarrow \texttt{d-} \rightarrow \texttt{dtack+}$$

shows that the transceiver is enabled early, allowing the device to observe valid bus data when the write request is asserted.

The READ and WRITE controllers were modelled separately to simplify verification. Isolating each mode reduces model complexity, making it easier to validate correctness and identify errors before integration.

In both STGs, limited concurrency appears during the reset phase. Some deassertion transitions, such as `lds-` and `ldtack-`, may occur concurrently with the next request (`dsr+` or `dsw+`). This overlap allows a new transaction to begin while the previous one is still completing its reset, improving overall responsiveness.

IMAGE SHOWING THE VME READ STG HERE.

IMAGE SHOWING THE TRACE OF THE VME READ STG HERE.

IMAGE SHOWING THE TIMING DIAGRAM OF THE VME READ STG HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR THE VME READ STG HERE.

This covers the read controller. The write controller follows similarly:

IMAGE SHOWING THE VME WRITE STG HERE.

IMAGE SHOWING THE TRACE OF THE VME WRITE STG HERE.

IMAGE SHOWING THE TIMING DIAGRAM OF THE VME WRITE STG HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR THE VME WRITE STG HERE.

Part 2, Combine Read and Write Controllers, Procedure and Theory

A single controller must support both read and write operations, requiring the READ and WRITE STGs to be combined into a unified specification. The merged model must enforce mutual exclusion, ensuring that only one request (`dsr+` or `dsw+`) can occur at a time, while correctly sharing common signals (`lds`, `ldtack`, `dtack`, `d`). Each mode must also preserve its original sequencing constraints.

The combined STG uses a shared initial state in which the controller waits for either a read or a write request. This introduces a choice structure: from the initial place, either the READ or WRITE sequence may be taken, but not both. After executing the selected path, the controller returns to the shared initial state, ready for the next operation.

To construct the combined model, a new STG was created in Workcraft and saved as `vme-read_write.stg.work`. The READ and WRITE STGs were copied into this workspace and positioned nearby. The initial places of both models (e.g. `p1r` and `p1w`) were selected and merged using `Transformations → Merge selected places`, forming a single shared place `p1`. Any additional places representing shared states, such as those preceding `dtack-`, were merged in the same manner. The resulting STG clearly shows a choice from `p1`, where either `dsr+` or `dsw+` may fire.

Validation was performed using the simulation tool (`M`) to trace both read and write executions. Standard verification checks were also applied to confirm correctness of the combined model.

Part 2, Results and Discussion

Composing the READ and WRITE controllers is necessary because a single physical circuit must support both operations. Synthesis tools also require one unified specification, and composition ensures that the controller correctly arbitrates between read and write requests.

The composed model offers several benefits. A single implementation is obtained instead of two separate controllers, allowing hardware resources for shared signals such as `lds` and `dtack` to be reused. Mutual exclusion is enforced by the choice structure in the STG, guaranteeing that only one mode is active at any time. At the same time, concurrency is preserved, as the merged STG allows new requests to arrive during reset phases.

However, composition also introduces challenges. The combined STG is larger and more difficult to understand and verify. Merging behaviours can introduce CSC conflicts, where different execution paths lead to identical states that require different outputs, an issue explored further in Task 3. Increased STG complexity also leads to more complex synthesized circuits.

A key observation is the distinction between free and controlled choice. The merged initial place `p1` represents a free choice, where the environment selects either a read or write operation. Any subsequent merges represent controlled choices, where behaviour is determined by the earlier branch selection.

Finally, the model highlights an important aspect of VME operation. Reset signals from the device, such as `lds-` and `ldtack-`, can occur concurrently with new bus requests (`dsr+` or `dsw+`). This overlap allows a new transaction to begin before the previous one has fully completed, enabling pipelined operation and improved bus throughput.

IMAGE SHOWING THE VME READ-WRITE COMBINED STG HERE.

IMAGE SHOWING THE TRACE OF THE VME READ-WRITE COMBINED STG HERE.

IMAGE SHOWING THE TIMING DIAGRAM OF THE VME READ-WRITE COMBINED STG HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR THE VME READ-WRITE COMBINED STG HERE.

Part 3, CSC Conflict Verification, Theory and Procedure

Complete State Coding (CSC) is a key requirement for correct circuit synthesis. It ensures that every distinct state in an STG has a unique binary encoding defined by the values of its signals. A CSC conflict occurs when two different STG states share identical signal values but require different output transitions to occur next.

This situation is problematic because the circuit can only observe current signal values, not the execution history. If two states appear identical yet demand different behaviour, the circuit cannot determine which action to take.

CSC conflicts arise naturally in the merged VME controller. Both READ and WRITE modes reuse the same set of signals, and at certain points their executions may produce identical signal values. Despite this, the required next action can differ between modes. For example, in the READ path the transceiver may need to be enabled (`d+`) after `ldtack+`, whereas in the WRITE path `d+` has already occurred earlier.

To identify these conflicts, the merged model `vme-read_write.stg.work` was opened in Workcraft and checked using `Verification → Complete State Coding (all cores) [MPSat]`. The resulting report highlights the states that violate the CSC property.

Part 3, Results and Discussion

CSC verification of the merged controller fails, and the tool reports one or more CSC conflicts. A screenshot of the conflict report is included.

Each reported conflict consists of a pair of STG states reached via different execution traces, for example one through the READ path and the other through the WRITE path. Although these states have identical values for all signals, they enable different output transitions. As a result, the circuit cannot determine which action to take based solely on the observed signal values.

In the VME controller, such conflicts typically arise during the reset phase, where signal combinations may repeat across both modes. While the current signal values are identical, the required behaviour depends on the execution history, specifically whether the controller arrived from a read or a write operation. This historical information is not visible to the circuit.

This issue is fundamental to circuit synthesis. A combinational circuit computes outputs as a function of its current inputs. If two distinct situations present the same inputs but require different outputs, no valid combinational function exists. To resolve this ambiguity, additional state must be introduced to distinguish between the conflicting

situations. In practice, this is achieved by adding internal signals to encode the missing state information.

IMAGE SHOWING THE VERIFICATION RESULTS FOR CSC CONFLICTS IN VME CONTROLLER HERE.

IMAGE SHOWING THE CSC CONFLICT DETAILS FOR VME CONTROLLER HERE.

TERMINAL OUTPUT SHOWING DETAILS

Part 4, Resolving CSC Conflicts, Procedure and Theory

CSC conflicts must be resolved before circuit synthesis. The general solution is to introduce additional internal signals that encode extra state information, allowing the circuit to distinguish between states that previously appeared identical.

One approach is **automatic resolution using Petrify**. Petrify (via MPSat) can insert internal signals automatically to eliminate CSC conflicts. This is done using `Tools → Encoding conflicts → Resolve CSC conflicts [Petrify]`. As a result, a new internal signal (typically named `csc0`) is added, with transitions positioned to differentiate the conflicting states.

CSC conflicts may also be resolved **manually** by inserting internal signals based on an understanding of the model. The read and write paths are first identified, and an internal signal is added such that it takes different values on each path. Transitions are placed so that the conflicting states no longer share identical signal encodings.

A structured manual approach is the **state signal method**. Here, a dedicated internal signal (e.g. `mode`) explicitly records whether the controller is operating in read or write mode. For example, `mode+` may fire after `dsr+` to indicate read mode, while `mode-` fires after `dsw+` to indicate write mode. This directly encodes the branch selection.

Finally, Petrify may apply an **arbitrary signal insertion** strategy. The tool analyses the conflicting states, determines minimal signal placements, and inserts internal signals that resolve the conflicts. These signals may not have an intuitive interpretation, but they are sufficient to guarantee correct and conflict-free synthesis.

Part 4, Results and Discussion

EXPLAINING THE DIFFERENCE BETWEEN THE 3+ RESOLUTION METHODS LATER ON.

IMAGES TO SHOW THE BS THAT WE NEED LATER ON, NEED TO SHOW THE DIFFERENT RESOLUTIONS, MAPPING AND ETC. INCLUDE VERIFICATION RESULTS.

Part 5, Synthesis and Verification, Theory and Procedure

Once CSC conflicts are resolved, the STG can be synthesised into a circuit. Synthesis proceeds by generating the full state graph of the STG, deriving next-state logic for each output and internal signal, and mapping the resulting Boolean equations to logic gates, either as complex gates or through technology mapping.

To perform synthesis, the CSC-resolved STG was used and processed via `Synthesis →`

`Complex gate [MPSat]` (or technology mapping). The resulting circuit was saved and functionally validated using the simulation tool (`M`) by exercising both read and write sequences.

Formal verification was then applied to ensure correctness of the synthesised circuit. The environment property was set to the original specification `vme-read_write.stg.work`, rather than the CSC-resolved model. Conformation, deadlock freeness, and output persistency were verified using MPSat, either individually or using the combined verification option with unfolding reuse.

Part 5, Results and Discussion

Results & Discussion

Screenshots of the synthesised circuit and the verification results are included. All verification checks pass.

Inspection of the circuit shows the logic implementing each output signal (`lds`, `dtack`, and `d`), as well as the internal signal `csc0` introduced during CSC resolution. Feedback paths are visible in the circuit, where outputs feed back into gate inputs, providing the memory required to store state.

Verification is performed against the original STG rather than the CSC-resolved model because the original STG represents the behavioural contract with the environment. CSC resolution only introduces internal implementation details. Verifying against the original specification ensures that the synthesised circuit preserves the intended external behaviour, independent of how internal state is encoded.

Each verification check addresses a different correctness aspect. Conformation ensures that the circuit produces correct outputs for all legal input sequences defined by the STG. Deadlock freeness guarantees that the circuit can always make progress and never becomes stuck waiting indefinitely. Output persistency ensures hazard-free behaviour, meaning that once an output is enabled it completes its transition without glitches.

Results & Discussion

Screenshots of the synthesised circuit and the verification results are included. All verification checks pass.

Inspection of the circuit shows the logic implementing each output signal (`lds`, `dtack`, and `d`), as well as the internal signal `csc0` introduced during CSC resolution. Feedback paths are visible in the circuit, where outputs feed back into gate inputs, providing the memory required to store state.

Verification is performed against the original STG rather than the CSC-resolved model because the original STG represents the behavioural contract with the environment. CSC resolution only introduces internal implementation details. Verifying against the original specification ensures that the synthesised circuit preserves the intended external behaviour, independent of how internal state is encoded.

Each verification check addresses a different correctness aspect. Conformation ensures that the circuit produces correct outputs for all legal input sequences defined by the STG. Deadlock freeness guarantees that the circuit can always make progress and never

becomes stuck waiting indefinitely. Output persistency ensures hazard-free behaviour, meaning that once an output is enabled it completes its transition without glitches.

IMAGE SHOWING THE SYNTHESISED VME CONTROLLER CIRCUIT HERE.

IMAGE SHOWING THE VERIFICATION RESULTS FOR THE SYNTHESISED VME CONTROLLER CIRCUIT HERE.

GATE MAPPINGS AND ETC HERE.

Task 6, Verilog to VHDL Translation

The synthesised circuit can be exported to Verilog for use in standard digital design flows. The task requires translating this to VHDL and explaining the connection to previous lab tools.

References

Appendix